ResiDual: Transformer with dual residual connections[★]

Shufang Xie^a, Huishuai Zhang^b, Junliang Guo^c, Xu Tan^{c,*}, Jiang Bian^c,
Hany Hassan Awadalla^c, Arul Menezes^c, Tao Qin^d, Rui Yan^{e,*}

^a Gaoling School of Artificial Intelligence, Renmin University of China, No.59, Zhongguancun Street, Haidian District, Beijing, 100872, China

^b Wangxuan Institute of Computer Technology, Peking University, No. 128 Zhongguancun North Street, Haidian District, Beijing, 100871, China

^c Microsoft Research, No. 5 Danling Street, Haidian District, Beijing, 100080, China

^d Zhongguancun Academy, Haidian Dayue Information Technology Park, No. 17, Second Ring Road, Daniufang, Haidian District, Beijing, 100094, China

^e School of Artificial Intelligence, Wuhan University, Luojia Hill, Wuchang District, Wuhan, 430072, China

ARTICLE INFO

Keywords:

Transformers

Residual connections

ABSTRACT

Transformers rely heavily on residual connections for stable and effective training. Existing approaches, however, face distinct drawbacks. Post-Layer Normalization (Post-LN) Transformers suffer from gradient vanishing in deep networks. Pre-Layer Normalization (Pre-LN) Transformers, by contrast, encounter representation collapse, limiting model expressiveness. To overcome both limitations simultaneously, we propose ResiDual, a novel Transformer architecture that integrates dual residual connections inspired by both Post-LN and Pre-LN designs. Crucially, we provide rigorous theoretical analyses showing that ResiDual effectively mitigates gradient vanishing through a guaranteed gradient-norm lower bound inherited from the Pre-LN branch while maintaining representation diversity through the Post-LN branch. ResiDual introduces only modest computational overhead and scales effectively to very deep Transformer models (e.g., 50 layers) without instability or performance degradation. Extensive experiments further validate our theoretical insights, showing that ResiDual consistently outperforms both Post-LN and Pre-LN Transformers across multiple machine translation, language modeling, and image classification benchmarks, datasets, and network depths. Our results demonstrate that the simple yet principled fusion of dual residual connections can substantially improve Transformer performance and scalability.

1. Introduction

Transformer [1] has emerged as a powerful neural network architecture and achieved remarkable success across various tasks, including machine translation [1], language modeling [2,3], image recognition [4], and speech synthesis [5]. Despite this success, the optimal design of residual connections, an essential component crucial for stable training, remains an open and critical research question [6–8].

Existing Transformer architectures predominantly adopt two residual connection variants: Post-Layer Normalization (Post-LN) and Pre-Layer Normalization (Pre-LN). Post-LN applies Layer Normalization (LN) after each residual block and is widely employed in models such as BERT [9] and RoBERTa [10]. Pre-LN, instead, places LN before each residual block, as seen in GPT models [11]

[★] Code is available at: <https://github.com/microsoft/ResiDual>

^{*} Corresponding authors.

E-mail addresses: tanxu2012@gmail.com (X. Tan), rui.yan@whu.edu.cn (R. Yan).

Table 1
Comparison of Post-LN,
Pre-LN, and our method.

Method	GV	RC
Post-LN	☹️	☹️
Pre-LN	☹️	☹️
ResiDual	😊	😊

GV: gradient vanishing;
RC: representation collapse. 😊: no issue; ☹️: has issue.

and ViT [4]. However, both variants exhibit significant limitations (summarized in Table 1): Post-LN suffers from severe gradient vanishing in deep layers because gradients decay exponentially due to repeated normalization [7], whereas Pre-LN encounters representation collapse, in which hidden representations in higher layers become increasingly similar, drastically limiting the model’s expressiveness [6].

Several approaches have been proposed to alleviate these issues. Existing solutions generally fall into three categories: (1) architectural modifications adding aggregation or normalization modules (e.g., DLCL [12], NormFormer [13], RealFormer [14], B2T [15], Mix-LN [16]); (2) residual weighting techniques that adjust residual connection strength (e.g., Admin [6], DeepNet [17], τ -ResNet [18], ReZero [8]); and (3) advanced initialization strategies designed to stabilize training (e.g., T-Fixup [19], Foundation Transformer [20]). While these methods address either gradient vanishing or representation collapse individually, a unified solution capable of simultaneously resolving both issues remains unexplored.

In this paper, we introduce ResiDual, a novel Transformer architecture that leverages dual residual connections to inherit the advantages of both Post-LN and Pre-LN designs while effectively mitigating their respective shortcomings. Specifically, our ResiDual employs a carefully designed Pre-Post-LN (PPLN) structure (as illustrated in Fig. 1(c)), consisting of two residual branches: one branch resembling Pre-LN to ensure stable gradient flow, and another branch similar to Post-LN to preserve representation diversity. Importantly, our design is not merely a naive combination; rather, we provide rigorous theoretical justifications demonstrating that this dual residual structure inherently addresses the underlying theoretical issues of existing variants.

Theoretically, we first identify that the gradient vanishing problem persists even with adaptive optimizers such as Adam [21]. We then rigorously prove that our ResiDual architecture guarantees a bounded gradient norm due to the Pre-LN branch, thus effectively eliminating gradient vanishing. Additionally, we show analytically that ResiDual retains the representation diversity characteristic of Post-LN, thereby resolving the representation collapse issue encountered by Pre-LN.

Empirically, we validate our theoretical insights through extensive experiments across multiple tasks and domains. We evaluate our approach on representative machine translation benchmarks (small-scale IWSLT, mid-scale WMT, and large-scale OPUS), decoder-only language modeling on a 10B-token subset of OpenWebText together with downstream language understanding/reasoning benchmarks and Gigaword summarization fine-tuning, and image classification tasks using vision transformers. For machine translation, we evaluate translation quality with BLEU, ChrF + +, and COMET, train all models with multiple random seeds, and report statistical significance against a broad set of Transformer residual and normalization baselines. We also analyze efficiency in terms of throughput, peak memory, per-token FLOPs, and wall-clock time. Experimental results consistently show that ResiDual improves over strong standard baselines across the datasets and depths we study, with the advantage amplified in deeper models. We also verify the effectiveness of our model on very deep architectures (up to 50 layers) and observe consistently strong performance.

In summary, our contributions are threefold:

- We propose ResiDual, a novel Transformer architecture integrating dual residual connections, effectively addressing both gradient vanishing (Post-LN) and representation collapse (Pre-LN) issues.
- We provide rigorous theoretical analyses explaining why ResiDual can inherit the strengths of both residual variants without their corresponding weaknesses.
- We empirically validate our theoretical findings across diverse tasks and datasets, consistently demonstrating superior performance compared to existing Transformer residual designs.

2. Related work

Numerous studies have proposed ways to improve Transformer networks. From the perspective of normalization and residual design, these methods fall into three main categories. First, some works, such as DLCL [12] and B2T [15], modify the architecture itself. Specifically, DLCL [12] adds residual connections from all previous layers to the final layer, while B2T [15] introduces an additional path after every two layers. These modifications change the computational graph and thereby ease gradient vanishing. Meanwhile, Mix-LN [16] uses Post-LN in lower layers and Pre-LN in higher layers, while HybridNorm [22] uses QKV norms. Another variant, Gemma 2 [23], applies an additional Post-LN to each sub-layer’s output before residual addition. Second, some studies introduce a weight on the residual branch, such as Admin [6], DeepNet [17], ReZero [8], and GPT-2 [11]. This weighting helps control hidden-representation variance across layers and can alleviate some issues inherent to Transformers. Finally, some works

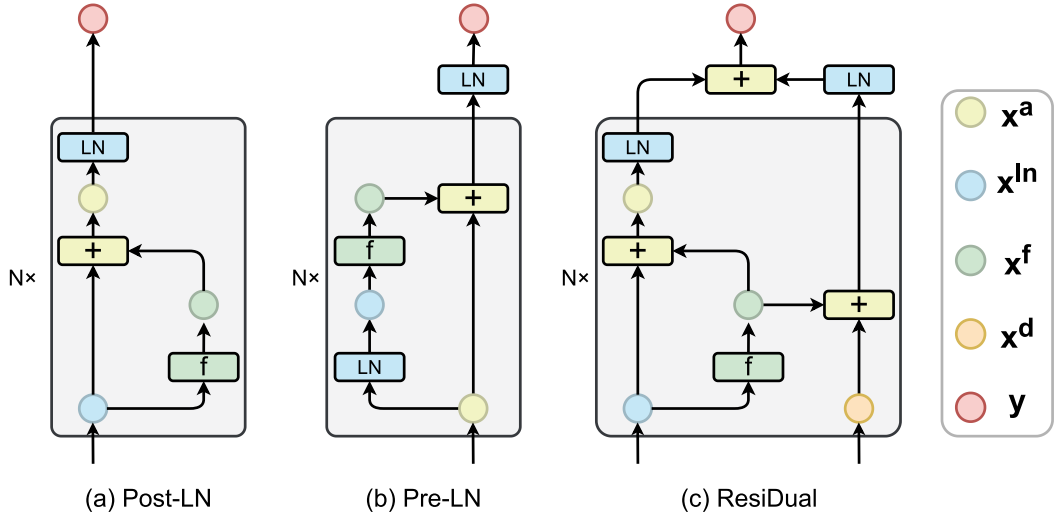


Fig. 1. Overview of Post-LN, Pre-LN, and ResiDual. Circles with different colors represent different variables and rectangles represent different operations. See Section 3 for more details about the notations.

modify the initialization, such as T-Fixup [19], DeepNet [17], and Foundation Transformer [20], to reduce variance and stabilize training. In this study, we propose a novel, simple, and effective method to address these issues, backed by both empirical and theoretical verification across multiple tasks.

Theoretical understanding of Transformer networks is an important research direction. Xiong et al. [7] were the first to uncover the imbalanced gradient distribution in the Post-LN variant, while Liu et al. [6] identified the representation bottleneck of Pre-LN Transformers. More recently, Wang et al. [17] examined Transformer optimization from the model-update perspective. In contrast to these studies, we investigate Transformer optimization with the Adam [21] optimizer, which is more widely used than SGD. We also provide a quantitative analysis of the convergence rate of representation collapse. We anticipate that our study will contribute to future research on this problem.

3. Method

3.1. Disadvantages of Post-LN and Pre-LN

In this section, we briefly review the architecture of Post-LN and Pre-LN, whose illustrations are available in Fig. 1(a) and (b). We will also discuss the shortcomings of each architecture.

Gradient Vanishes with Post-LN. The Post-LN architecture is shown in Fig. 1(a). More specifically, given a Post-LN Transformer network with N residual blocks, we assume the input shape is $n \times d$, where n and d denote the sequence length and embedding size, respectively¹. The variables with vector arrow (e.g., $\vec{x} \in \mathbb{R}^{n \times d}$) denote the whole sequence and the variables without it (e.g., $x \in \mathbb{R}^d$) denote an element of the sequence. We use $\vec{x}^a \in \mathbb{R}^{n \times d}$ to denote the tensor after the add operation and use subscript k (i.e., $\vec{x}_k^a$) to denote the tensor in the k -th block. We also use $\vec{x}_k^{ln} \in \mathbb{R}^{n \times d}$ to denote the normalized tensor and $\vec{x}_k^f \in \mathbb{R}^{n \times d}$ to denote the output of the function $f_k(\cdot; \mathbf{w}_k)$ in the k -th block. The f_k can be a self-attention, cross-attention, or feed-forward with parameter $\mathbf{w}_k$. Using these notations, the Post-LN computation of each element in the k -th block is

$$\begin{aligned} \mathbf{x}_k^a &= \mathbf{x}_k^{ln} + \mathbf{x}_k^f = \mathbf{x}_k^{ln} + f_k(\mathbf{x}_k^{ln}; \mathbf{w}_k); \\ \mathbf{x}_{k+1}^{ln} &= \text{LN}(\mathbf{x}_k^a). \end{aligned}$$

Finally, the output $\mathbf{y}$ is computed by $\mathbf{y} = \mathbf{x}_{N+1}^{ln} = \text{LN}(\mathbf{x}_N^a)$. Intuitively, $\mathbf{x}_k^f$ is normalized $N - k$ times, and so are the gradients of $\mathbf{w}_k$. Therefore, the gradients of lower blocks will be small. From [7], we know that for a Post-LN Transformer, the gradient norm decreases exponentially from deep layers to shallow layers. Such imbalanced gradients impede model training. Therefore, in practice, training tricks such as learning-rate warm-up are necessary to train a Post-LN model.

Representation Collapses with Pre-LN. With the same notations, the Pre-LN computation is

$$\mathbf{x}_k^{ln} = \text{LN}(\mathbf{x}_k^a);$$

¹ We omit the batch dimension that will not affect our analysis.

$$\mathbf{x}_{k+1}^a = \mathbf{x}_k^a + \mathbf{x}_k^f = \mathbf{x}_k^a + f_k(\bar{\mathbf{x}}_k^{ln}; \mathbf{w}_k).$$

Similarly, the model output is $\mathbf{y} = \text{LN}(\mathbf{x}_{N+1}^a) = \text{LN}(\sum_{k=1}^N \mathbf{x}_k^f)$. Intuitively, since $\mathbf{x}_k^f$ is only normalized once when computing $\mathbf{y}$, neither the forward nor the backward pass is blocked by LN. Thus, Pre-LN does not have the gradient vanishing issue. However, it suffers from another issue called representation collapse. More specifically, Liu et al. [6] show that $\frac{\sqrt{\text{Var}[\mathbf{x}_k^f]}}{\sqrt{\text{Var}[\mathbf{x}_k^a + \mathbf{x}_k^f]}}$ is likely to be smaller for higher blocks (i.e., blocks with larger k). This means the output of the later blocks ($\mathbf{x}_k^f$) has little contribution to the total variance of $\mathbf{x}_k^a$. In Section 4.3, we show that the difference between $\mathbf{x}_{k+1}^{ln}$ and $\mathbf{x}_k^{ln}$ (i.e., $|\mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln}|$) decays along with k , which indicates the input of the higher blocks will collapse to similar values. We also show that this issue may limit the capacity of the model.

3.2. ResiDual

The goal of our model is to inherit the advantages of both variants while avoiding their disadvantages. To achieve this goal, we use residual paths from both variants, as illustrated in Fig. 1(c). More specifically, the two residual connections are illustrated in the left and right vertical lines in the Figure. The left one, which is similar to the conventional Post-LN, is

$$\begin{aligned} \mathbf{x}_k^a &= \mathbf{x}_k^{ln} + \mathbf{x}_k^f = \mathbf{x}_k^{ln} + f_k(\bar{\mathbf{x}}_k^{ln}; \mathbf{w}_k); \\ \mathbf{x}_{k+1}^{ln} &= \text{LN}(\mathbf{x}_k^a). \end{aligned}$$

Meanwhile, the right residual, which is similar to the conventional Pre-LN, is formulated by

$$\mathbf{x}_{k+1}^d = \mathbf{x}_k^d + \mathbf{x}_k^f,$$

where $\mathbf{x}^d \in \mathbb{R}^{n \times d}$ is the tensor that denotes the dual residual, which is similar to $\mathbf{x}^a$ in the Pre-LN and allows the gradients to flow directly to each block.

Finally, the output $\mathbf{y}$ is computed by adding the representation of both residuals, which is

$$\mathbf{y} = \mathbf{x}_{N+1}^{ln} + \text{LN}(\mathbf{x}_{N+1}^d).$$

3.3. Discussion

In this section, we only provide an intuitive explanation of ResiDual; the mathematical analysis is presented in Section 4.

Avoiding the Gradient Vanishing. In ResiDual, the gradient of each block flows through both residual connections. Thus, even if the gradient coming from the Post-LN-like residual vanishes, gradients still flow through the Pre-LN-like residual. This prevents gradient vanishing. We provide the details of the lower bound on the gradient norm in Section 4.1.

Avoiding the Representation Collapse. Our Pre-LN-like residual only affects the model output and does not affect the input to each block. Therefore, the representation capacity matches that of a Post-LN model. Furthermore, because the final output of our model is the sum of two residual connections, the output representation does not collapse either. We provide the details of the lower bound on the representation capacity in Section 4.3.

Minimal Additional Computational Overhead. In Transformer architectures, the predominant computational load arises from the attention and feed-forward layers, which involve multiple matrix multiplications. Conversely, residual operations, i.e., tensor additions, are computationally much less demanding. Empirically, our WMT efficiency study shows only a small wall-clock difference (roughly 2–5%) relative to the standard Pre-LN/Post-LN baselines. Therefore, our method is a simple yet effective modification to the network.

4. Theoretical analysis of ResiDual

4.1. The gradient vanishing issue

In order to present the analysis concisely, we study a simple setting and make several assumptions. In Transformer models, the f function can be either a feed-forward block or a multi-head attention block. For a feed-forward block, $f(\mathbf{x}) := \mathbf{W}\mathbf{x}$ where we ignore the layer index. For a multi-head attention block, we have weight matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$. For simplicity, we focus on single-head attention. Similar to [7], we initialize $\mathbf{W}_Q$ to be zero matrices and consequently, the attention is a uniform distribution at initialization and $f(\mathbf{x}^{(j)}) := \frac{1}{n} \sum_{j=1}^n \mathbf{x}^{(j)} \mathbf{W}_V$ where we drop the layer index and $\mathbf{x}^{(j)}, j \in [n]$ are the input sequence with length n . We usually drop the superscript index $^{(j)}$ for notational simplicity when the context itself is clear. We introduce $\bar{\mathbf{x}} := \{\mathbf{x}^{(j)}, j \in [n]\}$ and use $\mathbf{w}$ to denote the collection of parameter matrices in f .

Based on the above assumptions, without loss of generality, we further assume that the f function keeps the norm, i.e., $\|f(\mathbf{x})\| = \|\mathbf{x}\|$. This assumption is asymptotically true when the network width goes to infinity and the initialization variance is properly scaled. We assume that the signal is standardized after Layer Normalization, i.e., $\|\mathbf{x}_k^{ln}\| = \sqrt{d}$ for all $k \in [N]$, and that for $\mathbf{x} \in \mathbb{R}^d$, the Jacobian matrix through LN satisfies $\frac{\partial \text{LN}(\mathbf{x})}{\partial \mathbf{x}} \approx \frac{\sqrt{d}}{\|\mathbf{x}\|_2} \mathbf{I}$. This approximation can be achieved if the mean of $\mathbf{x}$ is 0 and the variance is $\frac{1}{d} \|\mathbf{x}\|^2$ while

ignoring the gradient back-propagated through mean and variance. The rationale in this assumption is that the error signal (gradients) back-propagating through LN becomes smaller as the norm of the input to the LN gets larger. In the Post-LN Transformer, the scale of the inputs to the Layer Normalization is independent of N , and thus the gradients of parameters in the last layer are independent of N .

From [7], we know that for the Post-LN Transformer, the gradient norm of the block k decreases exponentially as block index k gets smaller. This indicates that the gradient of the block close to the input would be exponentially small for deep Transformers. In contrast, for the Pre-LN Transformer, the gradient norm of each block is roughly independent of the block index k . For completeness, we rephrase the result from [7] with our notations and assumptions. We also present the proof in a more explicit form in [Appendix B](#).

Theorem 1 (Gradients of the k -th block in the Post-LN and Pre-LN Transformers). *Given the above assumptions on f and LN, for the Post-LN Transformer with N blocks, the gradient of the parameters of the k -th block satisfies*

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right\|_F \approx \mathcal{O}\left((1/2)^{(N-k)/2} e^{\sqrt{N-k}}\right), \quad (1)$$

for the Pre-LN Transformer with N blocks, the gradient of the parameters of the k -th block satisfies

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right\|_F \approx \mathcal{O}\left(\sqrt{\frac{\log(N-k)}{N}}\right), \quad (2)$$

where $k < N$ and we ignore the terms irrelevant with k, N .

4.2. Analysis of Adam

It is worth noting that the gradient vanishing problem does not directly translate into inefficient training with Adam [21], because the actual update is rescaled to a normal magnitude even when extremely small gradients are obtained. However, the gradient vanishing problem still affects the stability of the Adam optimizer, as we argue below. More specifically, we show that the Adam update has a large condition number (i.e., is ill-conditioned) when gradients vanish.

To be more specific, let $\alpha, t, \epsilon, \beta_1, \beta_2$ denote the learning rate, training step, smoothing factor, first decay rate, and second decay rate, respectively. Let $\mathbf{w}^{(t)}, \mathbf{g}, \hat{\mathbf{m}}^{(t)}, \hat{\mathbf{v}}^{(t)}$ represent the parameters, gradients, and bias-corrected first and second moment estimates at time t . The Adam update is defined as

$$\mathbf{u}(\mathbf{g}^{(t)}) = \alpha \cdot \hat{\mathbf{m}}^{(t)} / (\sqrt{\hat{\mathbf{v}}^{(t)}} + \epsilon), \quad \mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \mathbf{u}(\mathbf{g}^{(t)}).$$

Since the update is element-wise, we also use $u(g)$ to denote the scalar function such that $\mathbf{u}(\mathbf{g}) = [u(g_1), u(g_2), \dots, u(g_d)]$.

With these notations, the absolute condition number $\hat{\kappa}$ can be defined as:

$$\begin{aligned} \hat{\kappa} &= \lim_{\delta \rightarrow 0} \sup_{\|\delta \mathbf{g}\| \leq \delta} \frac{\|\mathbf{u}(\mathbf{g} + \delta \mathbf{g}) - \mathbf{u}(\mathbf{g})\|}{\|\delta \mathbf{g}\|} \\ &= \sqrt{\sum_{i=1}^d \left(\frac{\partial u}{\partial g_i} \right)^2}, \end{aligned}$$

where $\delta \mathbf{g}$ denotes a random perturbation bounded by δ .

Theorem 2. *For vanished gradients ($\mathbf{g} = 0$) at an early stage (t is small), the absolute condition number of the Adam update is $\hat{\kappa} \approx \frac{\alpha \sqrt{d}}{\epsilon}$.*

Proof. When the gradient $\mathbf{g} = 0$, we have

$$\frac{\partial u}{\partial g} = \frac{\alpha(1 - \beta_1)}{(1 - \beta_1^t) \left(\epsilon + \sqrt{\frac{\beta_2 v_{t-1,j}}{1 - \beta_2^t}} \right)}$$

In the early stage (i.e., t is small), for the vanished gradient ($g_i = 0$), the absolute condition number $\hat{\kappa}$ is

$$\hat{\kappa} = \alpha \frac{1 - \beta_1}{1 - \beta_1^t} \sqrt{\sum_{i=1}^d \frac{1}{\left(\epsilon + \sqrt{\frac{\beta_2 v_i^{(t-1)}}{1 - \beta_2^t}} \right)^2}} \approx \frac{\alpha \sqrt{d}}{\epsilon}. \quad (3)$$

□

The comprehensive proof of [Theorem 2](#) is provided in [Appendix C](#). Intuitively, when a small noise $\|\delta \mathbf{g}\| \leq \delta$ is added to the gradient $\mathbf{g}$, the update change $\|\mathbf{u}(\mathbf{g} + \delta \mathbf{g}) - \mathbf{u}(\mathbf{g})\|$ could be significantly larger than the noise $\|\delta \mathbf{g}\|$. This is due to $\alpha \sqrt{d}$ being potentially much larger than ϵ , making the training unstable and susceptible to minor perturbations. This study aligns with the empirical findings by [17], which reveal that exploding gradients in higher layers are not the primary cause of Post-LN training difficulties. A visualization of $\hat{\kappa}$ is also provided in [Appendix C](#).

Given a fixed model with width d , there appear to be two potential ways to reduce $\hat{\kappa}$: increasing ϵ or decreasing α . However, the former approach is impractical as a large ϵ compromises the adaptiveness of an optimizer. Hence, in practice, researchers often resort to reducing the learning rate α (e.g., using learning rate warm-up) to mitigate this problem. In Section 5.6, we also empirically investigate the effect of learning rate warm-up on Transformer training.

In conclusion, the issue of gradient vanishing remains critical even when the model is trained with adaptive optimizers. We, therefore, propose addressing this problem from an architectural standpoint.

4.3. The representation collapse issue

The Representation Collapse in Pre-LN. The issue with the representation capability of Pre-LN was initially observed by [6]. In summary, the Pre-LN Transformer's hidden representation cannot be refined by deeper layers due to the normalization of layer outputs. In this work, we propose a novel analysis approach that directly examines the distribution of hidden state changes, represented by $|\mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln}|$, and output changes, denoted by $|\mathbf{y}_N - \mathbf{y}_{N-1}|$. Our new method offers a straightforward way to obtain quantitative results regarding the convergence rate.

Theorem 3. For Pre-LN, assume $\mathbf{x}_k^f \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ independently for all $k \in [N]$, we have $\mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln} \sim \mathcal{N}(0, \omega_k^2 \mathbf{I})$ where $\omega_k^2 = \frac{2}{\sqrt{k}(\sqrt{k-1} + \sqrt{k})}$.

Proof. As $\mathbf{x}_k^f \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$, we have $\mathbf{x}_k^a = \sum_{j=1}^{k-1} \mathbf{x}_j^f$ thus $\mathbf{x}_k^a \sim \mathcal{N}(0, (k-1)\sigma^2 \mathbf{I})$. For the normalization layer, we approximate its effect as follows, $\mathbf{x}_k^{ln} = \frac{\mathbf{x}_k^a}{\sqrt{k-1}\sigma}$. Then we have

$$\begin{aligned} \mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln} &= \frac{\mathbf{x}_{k+1}^a}{\sqrt{k}\sigma} - \frac{\mathbf{x}_k^a}{\sqrt{k-1}\sigma} \\ &= \frac{\sqrt{k-1} - \sqrt{k}}{\sqrt{k(k-1)}\sigma} \cdot \mathbf{x}_k^a + \frac{1}{\sqrt{k}\sigma} \cdot \mathbf{x}_k^f. \end{aligned}$$

We know that $\frac{\sqrt{k-1} - \sqrt{k}}{\sqrt{k(k-1)}\sigma} \cdot \mathbf{x}_k^a \sim \mathcal{N}(0, \frac{(\sqrt{k-1} - \sqrt{k})^2}{k} \mathbf{I})$ and $\frac{1}{\sqrt{k}\sigma} \cdot \mathbf{x}_k^f \sim \mathcal{N}(0, \frac{1}{k} \mathbf{I})$. Because $\mathbf{x}_k^a$ and $\mathbf{x}_k^f$ are independent, we have $\mathbf{a}_{k+1} - \mathbf{a}_k \sim \mathcal{N}(0, \omega_k^2 \mathbf{I})$ and $\omega_k^2 = \frac{2}{\sqrt{k}(\sqrt{k-1} + \sqrt{k})}$. $\square$

Corollary 1. For each coordinate i of $\mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln}$, we have $\mathbb{E}[|(\mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln})_i|] \sim O(\frac{1}{\sqrt{k}})$

Corollary 2. When adding an extra layer to a $N-1$ layer Pre-LN Transformer, the output difference $\mathbb{E}[|(\mathbf{y}_N - \mathbf{y}_{N-1})_i|] \sim O(\frac{1}{\sqrt{N}})$ for each coordinate i .

Proof. Given two Pre-LN Transformers with $N-1$ and N layers, we denote their output as $\mathbf{y}_{N-1}$ and $\mathbf{y}_N$, respectively. Then we have

$$\begin{aligned} \mathbf{y}_{N-1} &= \text{LN}(\mathbf{x}_{N-1}^a + \mathbf{x}_{N-1}^f) = \mathbf{x}_N^{ln} \\ \mathbf{y}_N &= \text{LN}(\mathbf{x}_N^a + \mathbf{x}_N^f) = \mathbf{x}_{N+1}^{ln} \\ |\mathbf{y}_N - \mathbf{y}_{N-1}| &= |\mathbf{x}_{N+1}^{ln} - \mathbf{x}_N^{ln}| \end{aligned}$$

From Corollary 1, we can prove that $\mathbb{E}[|(\mathbf{y}_N - \mathbf{y}_{N-1})_i|] \sim O(\frac{1}{\sqrt{N}})$. $\square$

From Corollary 1, we can see that the expectation of $|\mathbf{a}_{k+1} - \mathbf{a}_k|$ decreases to 0 as k increases to infinity with rate $1/\sqrt{k}$. This means that, as the number of layers increases, the inputs to later layers become similar to each other. Thus, the capacity of the later layers is not fully used because they cannot further refine the representations. Meanwhile, Corollary 2 shows that adding an extra layer to a deep Pre-LN Transformer has little impact on the output, since the extra layer also cannot refine the model outputs, and thus the model's capacity may not be fully utilized.

4.4. Analysis of ResiDual

ResiDual Does Not Suffer From Gradient Vanishing Issue. For the ResiDual architecture (Fig. 1(c)), we can view it as a combination of a Post-LN Transformer and a Pre-LN Transformer. Specifically, in the forward pass, ResiDual behaves exactly like Post-LN except that it adds a dual branch consisting of the normalized sum of all block outputs at the end. In the backward pass, the error signal propagates through both branches. We can explicitly write down the gradients at block k as follows

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right)_{\text{post}} + \left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right)_{\text{dual}}, \quad (4)$$

where $\left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right)_{\text{post}}$ denotes the gradient component from the Post-LN branch and $\left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right)_{\text{dual}}$ denotes the gradient component from the dual branch. Specifically,

$$\left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right)_{\text{post}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{N+1}^{ln}} \left(\prod_{l=k}^N \frac{\partial \mathbf{x}_{l+1}^{ln}}{\partial \mathbf{x}_l^{ln}} \frac{\partial \mathbf{x}_l^{ln}}{\partial \mathbf{x}_k^f} \right) \frac{\partial \mathbf{x}_k^f}{\partial \mathbf{w}_k}$$

$$= \frac{\partial \mathcal{L}}{\partial \bar{\mathbf{x}}_{N+1}} \left(\prod_{l=k}^N \left(\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_l^f}{\partial \bar{\mathbf{x}}_l^{ln}} \right) \frac{\partial \bar{\mathbf{x}}_l^{ln}}{\partial \bar{\mathbf{x}}_l} \right) \frac{\partial \bar{\mathbf{x}}_k}{\partial \mathbf{w}_k},$$

and

$$\begin{aligned} \left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right)_{dual} &= \frac{\partial \mathcal{L}}{\partial \bar{\mathbf{x}}_{N+1}} \left(\prod_{l=k+1}^N \frac{\partial \bar{\mathbf{x}}_{l+1}}{\partial \bar{\mathbf{x}}_l} \right) \frac{\partial \bar{\mathbf{x}}_{k+1}^f}{\partial \mathbf{w}_k} \\ &= \frac{\partial \mathcal{L}}{\partial \bar{\mathbf{x}}_{N+1}} \left(\prod_{l=k+1}^N \left(\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_l^f}{\partial \bar{\mathbf{x}}_l^{ln}} \frac{\partial \bar{\mathbf{x}}_l^{ln}}{\partial \bar{\mathbf{x}}_l} \right) \right) \frac{\partial \bar{\mathbf{x}}_k^f}{\partial \mathbf{w}_k}. \end{aligned}$$

We see that when k is small, the Pre-LN gradient component dominates, whereas when k is close to N , the Post-LN gradient component dominates. It is therefore reasonable to estimate the gradient norm of the k -th block in the ResiDual Transformer as follows,

$$\begin{aligned} \left\| \frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} \right\|_F &\approx \max \left\{ \mathcal{O} \left(\left(\frac{1}{2} \right)^{\frac{N-k}{2}} e^{\sqrt{N-k}} \right), \right. \\ &\quad \left. \mathcal{O} \left(\sqrt{\frac{\log(N-k)}{N}} \right) \right\}, \end{aligned}$$

where again we ignore the terms irrelevant with N, k . Therefore, the ResiDual architecture does not suffer gradient vanishing problem.

ResiDual Does Not Suffer From Representation Collapse Issue.. The Post-LN and ResiDual do not have the representation collapse issue. To be more specific, we have the following theorem:

Theorem 4. In Post-LN and ResiDual, assume $\mathbf{x}_k^f \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ independently for all $k \in [N]$, the $\mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln} \sim \mathcal{N}(0, \omega^2)$ where ω is not related to k .

Proof. As $\mathbf{x}_{k+1}^{ln} = \text{LN}(\mathbf{x}_k^a) = \text{LN}(\mathbf{x}_k^{ln} + \mathbf{x}_k^f)$, $\mathbf{x}_k^{ln} \sim \mathcal{N}(0, \mathbf{I})$, and $\mathbf{x}_k^f \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$, we have

$$\begin{aligned} \mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln} &= \frac{\mathbf{x}_k^{ln} + \mathbf{x}_k^f}{\sqrt{1 + \sigma^2}} - \mathbf{x}_k^{ln} \\ &= \frac{(1 - \sqrt{1 + \sigma^2})\mathbf{x}_k^{ln} + \mathbf{x}_k^f}{\sqrt{1 + \sigma^2}}. \end{aligned}$$

Thus, $\mathbf{x}_{k+1}^{ln} - \mathbf{x}_k^{ln} \sim \mathcal{N}(0, \omega^2)$ where $\omega^2 = 2 - 2\frac{\sqrt{1+\sigma^2}}{1+\sigma^2}$ and ω is not related to k . $\square$

Corollary 3. When adding an extra layer to a $(N - 1)$ -layer ResiDual Transformer, the output difference satisfies $\mathbb{E}[|(\mathbf{y}_N - \mathbf{y}_{N-1})_i|] \geq \sqrt{\frac{2}{\pi}} \omega$ for each coordinate i .

Proof.

$$\begin{aligned} \mathbf{y}_N - \mathbf{y}_{N-1} &= (\mathbf{x}_N^{ln} + \text{LN}(\mathbf{x}_{N+1}^d)) - (\mathbf{x}_{N-1}^{ln} + \text{LN}(\mathbf{x}_N^d)) \\ &= (\mathbf{x}_N^{ln} - \mathbf{x}_{N-1}^{ln}) + (\text{LN}(\mathbf{x}_{N+1}^d) - \text{LN}(\mathbf{x}_N^d)) \end{aligned}$$

The $\text{LN}(\mathbf{x}_{N+1}^d) - \text{LN}(\mathbf{x}_N^d)$ is also a zero-mean Gaussian distribution, which can be denoted as $\mathcal{N}(0, \hat{\omega}_N^2)$. Then we have $\mathbf{y}_N - \mathbf{y}_{N-1} \sim \mathcal{N}(0, \omega^2 + \hat{\omega}_N^2)$. Therefore,

$$\begin{aligned} \mathbb{E}[|(\mathbf{y}_N - \mathbf{y}_{N-1})_i|] &= \sqrt{\frac{2}{\pi}} \sqrt{\omega^2 + \hat{\omega}_N^2} \\ &\geq \sqrt{\frac{2}{\pi}} \omega \end{aligned}$$

$\square$

In contrast to [Corollaries 2, 3](#) suggests that the hidden representations in our model can be refined by deep layers, thereby circumventing the issue of representation collapse.

Gradient norm and hidden representation visualization. To visualize the gradient norm and hidden representations across different layers, we constructed Transformer networks of 36 decoder layers using three distinct architectures: Pre-LN, Post-LN, and ResiDual. The configurations for embedding dimension, FFN dimension, and the number of attention heads were set to 256, 1024, and 4, respectively. We utilized the default initialization settings of fairseq [\[24\]](#). Subsequently, we created mini-batches consisting of 16 sentences and 20 tokens per sentence, and executed a forward and backward pass to gather the gradient norm and $\mathbf{x}_k^{ln}$ of each layer. The results are illustrated in [Fig. 2](#).

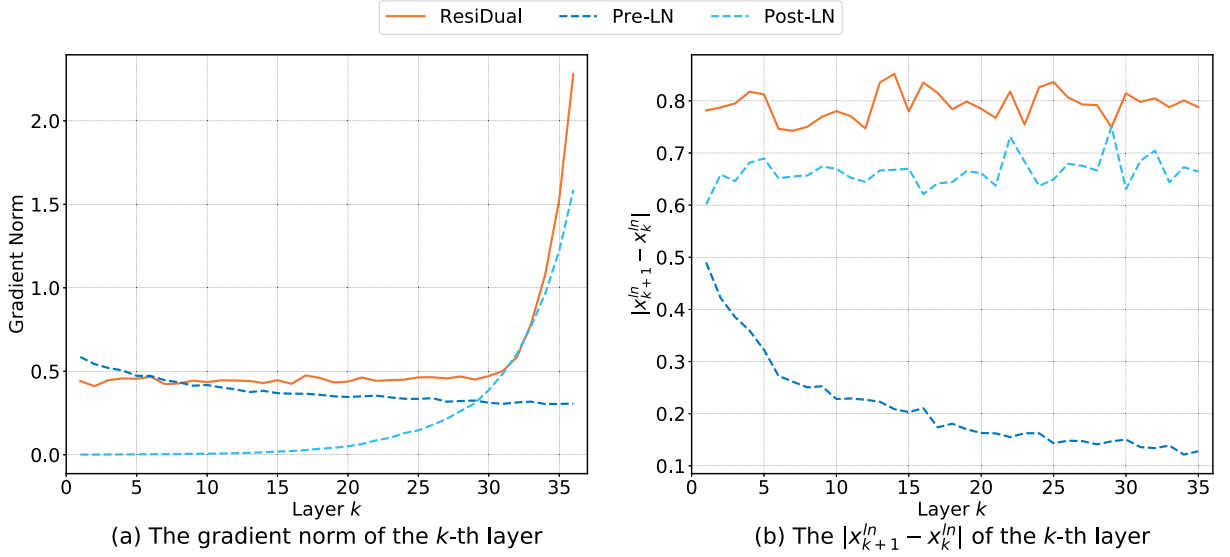


Fig. 2. Study of the gradient norm (left figure) and hidden representation (right figure) w.r.t. layer k . The solid lines represent our model and the dashed lines represent the Pre-LN and Post-LN models.

In Fig. 2(a), we present the gradient distribution of different methods. It is observed that the Post-LN demonstrates almost zero gradient for the earlier layers, an issue not encountered with the ResiDual (depicted by the orange line). This clearly indicates that our method can guarantee a lower-bound for the gradient norm. It is also important to note that none of the models exhibited the exploding-gradient problem. As per Theorem 1, the gradient of the final layer (i.e., $k = N$) is unrelated to N .

Fig. 2(b) displays the $|x_{k+1}^n - x_k^n|$ for the various architectures. As indicated by the lines, our method (the orange line) maintains a consistent value of $|x_{k+1}^n - x_k^n|$, whereas the value for Pre-LN decreases as the depth increases. These analyses suggest that the variance of $x_{k+1}^n - x_k^n$ does not decrease with increasing depth, allowing the later layers to continue refining the hidden representation. Furthermore, according to Corollary 3, the output of the model can also be refined with a lower bound that is not depth-related. In other words, ResiDual can circumvent the representation bottleneck often encountered in the Pre-LN model.

5. Experiments

5.1. Experiments on machine translation

We conducted experiments on the machine translation task, a representative task for the Transformer model, to evaluate our method. The experimental settings were as follows:

Data. We conducted experiments on three datasets: the IWSLT-14 German to English (DE→EN) dataset [25], the WMT English to German (EN→DE) dataset [26], and the OPUS-100 multilingual dataset [27]. The IWSLT dataset is relatively small, with 140k sentence pairs. We followed the scripts in Fairseq [24] to preprocess the data. The WMT dataset is larger, with 4M sentence pairs. We followed the preprocessing steps outlined in [28] by tokenizing the data with Moses tokenizer and then processing it with BPE [29]. The model was trained on the WMT-14 training set and evaluated on the test set from years 2010 to 2016, following [28]. The OPUS-100 dataset is a large-scale multilingual dataset containing 100 languages and approximately 55M sentence pairs (i.e., 4 billion tokens). We used the script from [27] to tokenize the data and used SentencePiece [30] to segment the tokens. Because we train our model in both directions (to and from English), the total training data is about 110M.

Implementation. All models are implemented using the Fairseq [24] framework, following conventional settings and initialization used in prior work. All experiments are executed on Nvidia Tesla V100 GPUs. We use ExDy to denote models with an x -layer encoder and a y -layer decoder. For instance, an E6D6 model represents a Transformer network with a six-layer encoder and a six-layer decoder, and E12D12 represents a Transformer network with a 12-layer encoder and a 12-layer decoder. In ResiDual and Pre-LN, there are occasions when the x_k^d may exceed the value range representable in FP16, potentially causing a training error. When this arises, a straightforward numerical trick is to downscale x_k^d to ensure that it remains within the FP16 range. This does not affect the results because $\text{LN}(x_{N+1}^d) = \text{LN}(\eta \cdot x_{N+1}^d)$ for any $\eta > 0$. Such an issue was not observed in FP32 training. Please refer to Appendix D.2 (Table D.11) for the hyper-parameter details.

Table 2

The experimental results on the IWSLT dataset for shallow (E6D6) and deep (E12D12) models.

Method	E6D6	E12D12
Pre-LN	35.12	35.18
Post-LN	35.37	Fail
DeepNet [17]	35.34	35.39
Admin [6]	35.50	35.67
T-Fixup [19]	34.88	35.45
NormFormer [13]	35.14	31.00
Mix-LN [16]	35.11	35.32
ResiDual	35.63	36.09

Table 3

The experimental results on the OPUS-100 dataset for different model depth. The star marker (*) denotes the results from [27] while others are produced by ourselves.

Method	#Layers	EX	XE	ALL
Pre-LN* [27]	6	21.4	27.5	24.5
	12	22.9	29.5	26.2
	24	24.0	31.4	27.7
Pre-LN	6	24.0	29.3	26.6
	12	25.4	30.8	28.1
	18	27.9	32.8	30.3
DeepNet [17]	100	29.0	33.2	31.1
ResiDual	6	25.1	30.7	27.9
	12	26.5	32.0	29.3
	18	28.7	33.4	31.0

Table 4

Multi-metric summary on the WMT dataset, averaged over 3 random seeds. We report mean $\pm$ standard deviation of BLEU, ChrF++ [31], and COMET [32] (wmt22-comet-da, scaled by 100) for both E6D6 and E18D18 models.

Method	E6D6			E18D18		
	BLEU $\uparrow$	ChrF++ $\uparrow$	COMET $\uparrow$	BLEU $\uparrow$	ChrF++ $\uparrow$	COMET $\uparrow$
Pre-LN	26.27 $\pm$ 0.16	52.90 $\pm$ 0.13	81.90 $\pm$ 0.10	26.90 $\pm$ 0.47	53.44 $\pm$ 0.07	82.82 $\pm$ 0.23
Post-LN	26.66 $\pm$ 0.07	53.01 $\pm$ 0.16	82.80 $\pm$ 0.09	Failed		
DLCL [12]	26.63 $\pm$ 0.13	53.10 $\pm$ 0.10	82.60 $\pm$ 0.08	26.71 $\pm$ 0.16	52.99 $\pm$ 0.00	82.66 $\pm$ 0.99
T-Fixup [19]	26.40 $\pm$ 0.05	52.94 $\pm$ 0.12	82.10 $\pm$ 0.18	26.69 $\pm$ 0.35	52.89 $\pm$ 0.06	81.60 $\pm$ 0.18
DeepNet [17]	26.57 $\pm$ 0.17	53.07 $\pm$ 0.09	82.59 $\pm$ 0.14	27.33 $\pm$ 0.32	52.89 $\pm$ 1.60	82.11 $\pm$ 2.41
Admin [6]	26.47 $\pm$ 0.05	53.01 $\pm$ 0.09	82.44 $\pm$ 0.04	27.27 $\pm$ 0.35	52.59 $\pm$ 2.03	81.59 $\pm$ 2.88
B2T [15]	26.61 $\pm$ 0.07	53.17 $\pm$ 0.11	82.64 $\pm$ 0.10	27.38 $\pm$ 0.27	52.22 $\pm$ 2.55	81.21 $\pm$ 3.28
RealFormer [14]	26.67 $\pm$ 0.07	53.00 $\pm$ 0.13	82.47 $\pm$ 0.08	27.41 $\pm$ 0.15	53.30 $\pm$ 0.10	83.43 $\pm$ 0.15
HybridNorm [22]	26.69 $\pm$ 0.11	53.00 $\pm$ 0.09	82.57 $\pm$ 0.09	25.48 $\pm$ 0.08	51.62 $\pm$ 0.94	79.76 $\pm$ 1.33
Gemma 2 [23]	26.24 $\pm$ 0.05	52.92 $\pm$ 0.09	81.99 $\pm$ 0.16	26.93 $\pm$ 0.07	53.21 $\pm$ 0.14	82.89 $\pm$ 0.37
ResiDual	26.82 $\pm$ 0.04	53.12 $\pm$ 0.09	82.80 $\pm$ 0.08	27.56 $\pm$ 0.08	53.66 $\pm$ 0.14	83.59 $\pm$ 0.10

Experimental Results. Table 2 presents the IWSLT results for shallow (E6D6) and deep (E12D12) models. The Post-LN method successfully converged for E6D6 but failed for E12D12. Conversely, the Pre-LN method converged in both settings but performed worse (35.12 and 35.18) than Post-LN at E6D6 (35.37) and our method at E6D6 (35.63). Moreover, methods such as DeepNet [17] and Admin [6] showed only slight improvements over the vanilla models, whereas our method achieved the best performance, notably a 0.9 BLEU point gain in E12D12 over the Pre-LN model. Preliminary experiments revealed that increasing model depth further led to overfitting because of the limited data, precluding the reporting of 18-layer model results.

We summarize the WMT results in Table 4, which reports the mean and standard deviation of three metrics at both depths: BLEU, ChrF++ [31], and COMET [32] (wmt22-comet-da, scaled by 100 for readability). ResiDual attains the best average BLEU at both E6D6 and E18D18, the best average COMET at E18D18 (tied with Post-LN at E6D6), the best ChrF++ at E18D18, and remains within 0.05 of the top ChrF++ score at E6D6. Taken together, the three metrics support the same overall conclusion, and ResiDual is also among the most stable methods across random seeds, especially in the deeper setting. By contrast, several baselines (e.g., Admin, B2T,

Table 5

The experimental results on WMT dataset (year 2010 to 2016) with E6D6 models. All results are averaged over 3 random seeds. Statistical significance of ResiDual over Pre-LN is assessed via bootstrap resampling [33,34]: * $p < .1$, ** $p < .05$, *** $p < .01$.

Method	2010	2011	2012	2013	2014	2015	2016	Average
Pre-LN	24.09±0.21	21.94±0.16	22.10±0.11	25.96±0.29	26.39±0.10	29.31±0.21	34.12±0.24	26.27±0.16
Post-LN	24.31±0.17	22.19±0.23	22.52±0.09	26.38±0.23	26.99±0.17	29.80±0.13	34.45±0.05	26.66±0.07
DLCL [12]	24.24±0.27	22.10±0.18	22.42±0.20	26.33±0.20	27.11±0.28	29.83±0.19	34.37±0.23	26.63±0.13
T-Fixup [19]	24.25±0.21	21.91±0.06	22.22±0.17	26.16±0.25	26.37±0.48	29.40±0.06	34.49±0.07	26.40±0.05
DeepNet [17]	24.36±0.29	21.96±0.18	22.42±0.32	26.38±0.44	26.89±0.08	29.88±0.26	34.12±0.58	26.57±0.17
Admin [6]	24.36±0.27	21.95±0.14	22.40±0.20	26.40±0.21	26.74±0.36	29.64±0.26	33.81±0.52	26.47±0.05
B2T [15]	24.22±0.14	22.01±0.11	22.49±0.18	26.50±0.20	26.90±0.09	29.67±0.26	34.45±0.24	26.61±0.07
RealFormer [14]	24.32±0.10	22.19±0.16	22.46±0.03	26.46±0.08	27.01±0.05	30.06±0.07	34.16±0.34	26.67±0.07
HybridNorm [22]	24.39±0.09	22.07±0.12	22.67±0.08	26.45±0.25	26.89±0.16	29.90±0.12	34.42±0.73	26.69±0.11
Gemma 2 [23]	24.17±0.06	21.87±0.15	22.20±0.09	26.16±0.15	26.32±0.06	29.21±0.32	33.76±0.08	26.24±0.05
ResiDual	24.42***±0.08	22.25*±0.06	22.67***±0.08	26.48*±0.15	27.43***±0.21	30.21***±0.01	34.29**±0.40	26.82±0.04

Table 6

The experimental results on WMT dataset (year 2010 to 2016) with E18D18 models. All results are averaged over 3 random seeds. Statistical significance of ResiDual over Pre-LN is assessed via bootstrap resampling [33,34]: * $p < .1$, ** $p < .05$, *** $p < .01$.

Method	2010	2011	2012	2013	2014	2015	2016	Average
Pre-LN	24.51±0.62	22.26±0.40	22.62±0.31	26.60±0.45	27.61±0.35	30.24±0.70	34.50±0.47	26.90±0.47
Post-LN	–	–	–	–	–	–	–	Fail
DLCL [12]	24.13±0.06	22.34±0.14	22.47±0.31	26.50±0.08	27.42±0.48	30.09±0.13	34.05±0.06	26.71±0.16
T-Fixup [19]	24.31±0.20	22.05±0.35	22.57±0.28	26.36±0.30	27.14±0.81	29.85±0.40	34.58±0.16	26.69±0.35
DeepNet [17]	24.60±0.55	22.62±0.19	23.03±0.35	27.11±0.26	28.19±0.56	30.96±0.35	34.80±0.51	27.33±0.32
Admin [6]	24.79±0.23	22.57±0.35	22.91±0.29	26.94±0.40	28.12±0.30	30.83±0.41	34.71±0.75	27.27±0.35
B2T [15]	24.74±0.17	22.64±0.15	22.94±0.29	26.95±0.43	28.36±0.71	30.96±0.51	35.04±0.10	27.38±0.27
RealFormer [14]	24.60±0.09	22.77±0.11	23.14±0.06	27.08±0.25	28.43±0.19	31.08±0.30	34.77±0.49	27.41±0.15
HybridNorm [22]	23.59±0.15	21.20±0.11	21.72±0.03	25.26±0.06	25.19±0.25	28.29±0.13	33.10±0.16	25.48±0.08
Gemma 2 [23]	24.43±0.31	22.27±0.18	22.72±0.06	26.62±0.06	27.81±0.04	30.37±0.03	34.31±0.05	26.93±0.07
ResiDual	24.81**±0.20	22.89**±0.13	23.26**±0.14	27.46***±0.18	28.44**±0.31	31.23*±0.15	34.84±0.36	27.56±0.08

DeepNet) display substantially larger across-seed variation in ChrF++ and COMET at E18D18, consistent with training instability in the deep regime. Per-year breakdowns of ChrF++ and COMET are deferred to [Appendix F \(Table F.15, F.16, F.17, F.18\)](#).

The per-year BLEU breakdowns are reported in [Tables 5 and 6](#) for the shallow (E6D6) and deep (E18D18) models, respectively. All models were trained with 3 random seeds and we report mean BLEU scores. Statistical significance of ResiDual over Pre-LN is assessed via bootstrap resampling [33,34]. In the tables, each row represents a different model and each column represents a different WMT test set, with the last column showing the average BLEU score and standard deviation across seeds. The Post-LN model converged only in the E6D6 setting, while the Pre-LN model showed convergence in both E6D6 and E18D18. Although Pre-LN benefits from the increased depth (26.27 → 26.90), it still lags behind our method at both depth configurations. We additionally compare with RealFormer [14], HybridNorm [22], and a variant employing the normalization technique from Gemma 2 [23]. In the E6D6 setting, HybridNorm (26.69) and RealFormer (26.67) are competitive, but our method achieves the best average BLEU (26.82) with the lowest across-seed standard deviation (0.04). In the E18D18 setting, the advantage of ResiDual becomes more pronounced: our method reaches 27.56 average BLEU while retaining a small standard deviation (0.08). Our method achieves the best average performance for both shallow and deep models, with statistically significant improvements over Pre-LN on most test sets. These results suggest that our method can consistently improve performance across model depths while maintaining training stability.

On the OPUS-100 dataset, which includes 100 language pairs and approximately 110M sentence pairs, our method also performs strongly. Because the Post-LN model failed to converge, [Table 3](#) reports only the Pre-LN results for this baseline family. The star marker (*) denotes the results from [27], while the others are produced by ourselves. Due to computational constraints, we report results only up to an 18-layer encoder and 18-layer decoder. We find that as model depth increases, our model continuously achieves better results. Meanwhile, for each depth, our model consistently outperforms the baselines, achieving a 0.7 to 1.2 BLEU point improvement over the standard Pre-LN model. Furthermore, the BLEU score is comparable to that of a 100-layer DeepNet [17] model, despite our model being more than five times shallower, highlighting its efficient utilization of deeper layers.

5.2. Language modeling

We further evaluate our method on autoregressive language modeling and downstream transfer to demonstrate its versatility across tasks and training objectives. Specifically, we train decoder-only Transformers of 12 and 24 layers with a hidden dimension of 768 on a 10B-token subset of OpenWebText [35]. Full model architectures and training configurations are provided in [Appendix E \(Table E.13\)](#).

We compare ResiDual against several strong baselines: Pre-LN [7], DeepNorm [17], RealFormer [14], HybridNorm [22], and a variant adopting the normalization scheme of Gemma 2 [23]. Post-LN failed to converge in this setting and is therefore reported in [Table 7](#) only as a failure case. In addition to validation perplexity (PPL) on the OpenWebText held-out set, we evaluate downstream perfor-

Table 7

Validation perplexity (PPL, lower is better) on the OpenWebText held-out set; accuracy (% , higher is better) on five downstream language understanding/reasoning benchmarks; and ROUGE (% , higher is better) on Gigaword summarization fine-tuning. The improvement over Pre-LN is highlighted in **bold** (for PPL, a negative value indicates improvement).

Model	PPL (↓)	Language Understanding / Reasoning						Gigaword Summ. (SFT)		
		OpenBookQA (0-shot)	CommonsenseQA (0-shot)	PIQA (5-shot)	Winogrande (10-shot)	ARC-Easy (10-shot)	AVG	R-1	R-2	R-L
12-layer Models										
Pre-LN	36.91	25.2	20.6	49.8	48.9	22.8	33.4	29.7	13.3	27.9
Post-LN	Fail	Fail	Fail	Fail	Fail	Fail	Fail	Fail	Fail	Fail
DeepNorm [17]	36.65	25.6	19.0	48.9	49.9	23.1	33.3	30.7	13.8	28.9
Gemma 2 [23]	37.09	22.8	20.1	50.7	51.2	25.6	34.1	30.9	14.1	29.0
RealFormer [14]	40.46	23.6	19.2	49.8	50.1	24.5	33.4	30.5	13.6	28.6
HybridNorm [22]	38.68	23.4	21.5	49.6	50.9	23.8	33.8	30.7	13.8	28.9
ResiDual	36.49 (-0.42)	26.6 (+ 1.4)	21.5 (+ 0.9)	50.4 (+ 0.6)	50.7 (+ 1.8)	24.2 (+ 1.4)	34.7 (+ 1.2)	31.4 (+ 1.7)	14.4 (+ 1.1)	29.6 (+ 1.7)
24-layer Models										
Pre-LN	33.84	23.6	20.1	50.5	50.0	24.9	33.8	32.1	15.6	30.0
Post-LN	Fail	Fail	Fail	Fail	Fail	Fail	Fail	Fail	Fail	Fail
DeepNorm [17]	32.29	22.0	20.1	48.9	50.5	25.8	33.5	32.3	15.7	30.2
Gemma 2 [23]	32.52	21.2	19.8	48.6	49.9	24.1	32.7	32.6	15.8	30.5
RealFormer [14]	34.36	24.0	21.0	50.0	50.2	25.3	34.1	32.3	15.1	30.4
HybridNorm [22]	35.84	25.4	20.4	50.1	51.3	23.7	34.2	32.4	15.6	30.2
ResiDual	31.93 (-1.91)	27.2 (+ 3.6)	21.4 (+ 1.2)	50.5 (+ 0.0)	50.6 (+ 0.6)	25.0 (+ 0.2)	34.9 (+ 1.1)	33.3 (+ 1.2)	16.0 (+ 0.4)	31.3 (+ 1.3)

mance on five representative language understanding/reasoning benchmarks: OpenBookQA [36], CommonsenseQA [37], PIQA [38], Winogrande [39], and ARC-Easy [40]. We also fine-tune the pretrained models on Gigaword summarization [41,42] and report ROUGE-1, ROUGE-2, and ROUGE-L.

The validation perplexity on the OpenWebText held-out set and the downstream evaluation results are summarized in Table 7. For compactness, we report five representative understanding/reasoning benchmarks together with a separate Gigaword summarization block. ResiDual consistently outperforms the baselines on intrinsic evaluation, downstream understanding/reasoning accuracy, and summarization fine-tuning. In the 12-layer setting, ResiDual attains the lowest PPL (36.49, improving over Pre-LN by 0.42), the best average understanding/reasoning accuracy (34.7, improving over Pre-LN by 1.2 points), and the best Gigaword results across all three ROUGE metrics (31.4 / 14.4 / 29.6 for ROUGE-1 / ROUGE-2 / ROUGE-L). The advantage is further amplified in deeper architectures: for the 24-layer models, ResiDual reaches a PPL of 31.93, a 1.91-point improvement over Pre-LN and the best among all methods, together with the best average understanding/reasoning accuracy (34.9, +1.1 over Pre-LN) and the strongest Gigaword summarization scores (33.3 / 16.0 / 31.3). The gains are particularly pronounced on tasks such as OpenBookQA (+3.6 points over Pre-LN at 24 layers), and the Gigaword results further show that the benefit of ResiDual extends beyond next-token prediction and in-context evaluation to supervised downstream fine-tuning, thereby broadening the validated scope of the architecture.

5.3. Experiments on image classification

Experimental Settings. In this study, we used the architecture introduced by [4], which is an encoder-only Transformer. We focused on two specific model variants, ViT-B/16 and ViT-B/32, which both have 12 layers but different patch sizes. While standard ViT models employ Pre-LN transformers, we adapted these models by integrating our novel architecture. These modified models are referred to as RD-B/16 and RD-B/32, respectively. Each model was trained from scratch, utilizing the CIFAR10 and CIFAR100 datasets, augmented with the AutoAugment technique as per [43]. We report the validation accuracy of each dataset across the training steps. Comprehensive hyper-parameters are provided in Appendix D.2 (Table D.12).

Experimental Results. The experimental outcomes are illustrated in Fig. 3. The top two figures present the results for CIFAR-10, while the bottom two depict those for CIFAR-100. On the left, we show the performance with patch size 16, whereas the right side shows the results with patch size 32. The baseline models (i.e., ViT-B/16 and ViT-B/32) are indicated by blue dashed lines, while our models (i.e., RD-B/16 and RD-B/32) are represented with red solid lines. The x-axis in each graph represents the training steps, and the y-axis indicates the top-1 validation accuracy. Across all four graphs, it is evident that the RD variant consistently surpasses the performance of the standard Pre-LN ViT during training. This outcome not only underscores the efficacy of our method in the vision domain but also, because ViT is an encoder-only Transformer, demonstrates the applicability of ResiDual beyond encoder-decoder models.

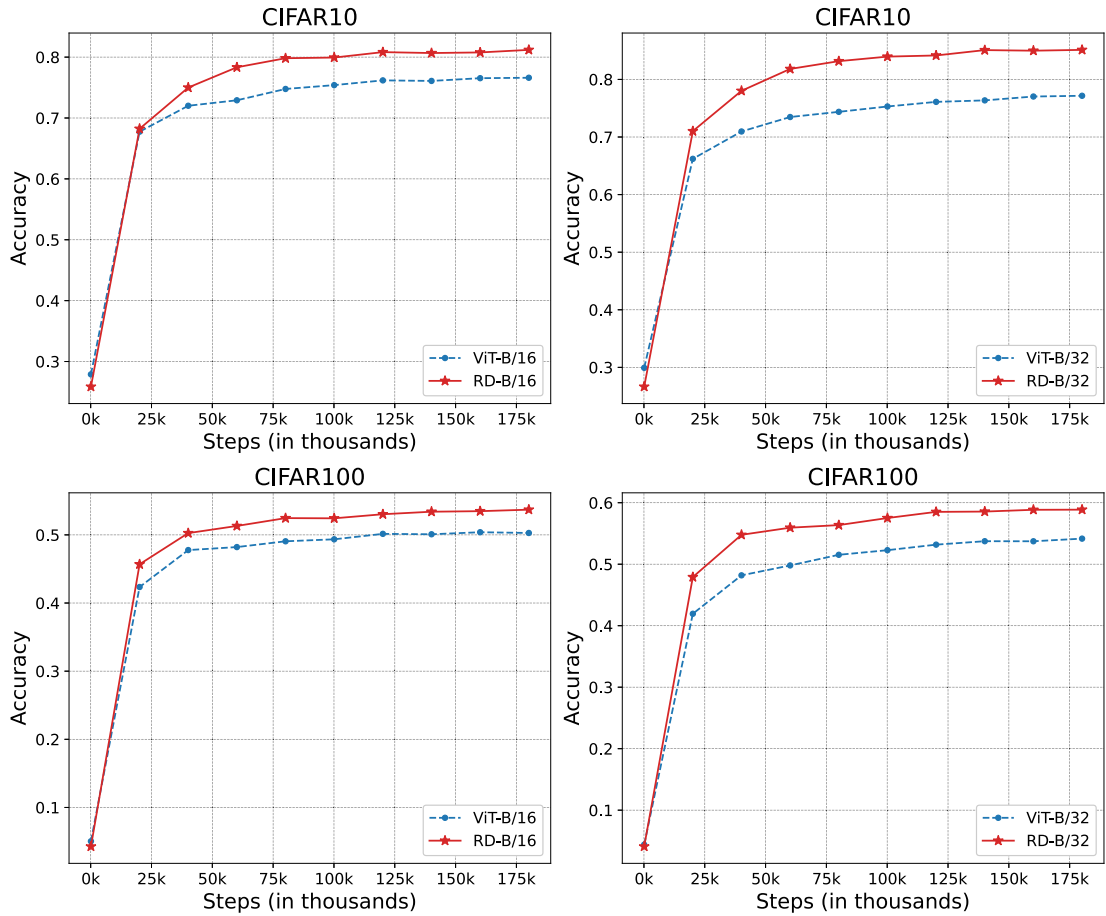


Fig. 3. Top-1 validation accuracy for CIFAR10 (top) and CIFAR100 (bottom) w.r.t. training steps. The red and the blue lines represent our method and baseline models, respectively. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

5.4. Exploring deeper model architectures

In our investigation into the effects of increasing model depth, we employed the E50D50 configuration, training these models on the WMT dataset. To control the total number of parameters and computational budget, we adjusted the dimensions of the embedding and the FFN, reducing them from 512 and 2048 to 256 and 1024, respectively. Other settings are identical to the experiments in Section 5.1.

The experimental results are presented in Fig. 4. In this figure, each column corresponds to a test set where we contrast our method with the Pre-LN baselines. The Post-LN baselines are omitted because they fail to converge. Although both Pre-LN and our method converge, our method consistently outperforms the Pre-LN variant across all test sets. On the test set with the largest gap, the BLEU score improves from 26.17 to 27.24. On average, the BLEU score improves from 26.30 to 26.91, indicating that our model can make more effective use of greater depth. This is consistent with the analysis in Section 4.3, which suggests that our model can avoid representation collapse in much deeper networks.

5.5. Performance and efficiency analysis

To quantify the computational overhead introduced by Residual, we conduct a comprehensive efficiency analysis. Table 8 reports the training throughput (tokens/sec and steps/sec), wall-clock time per epoch, peak per-GPU memory, and theoretical forward-pass FLOPs per token for both shallow (E6D6) and deep (E18D18) models on the WMT dataset. The counting conventions used to estimate these FLOPs follow standard practice [1,44,45]; the full derivation is deferred to Appendix A.

As shown in Table 8, all evaluated methods exhibit highly similar efficiency metrics within each depth block. This is expected: the primary differences among these variants, such as rearranging Layer Normalization, adjusting residual connections, or applying

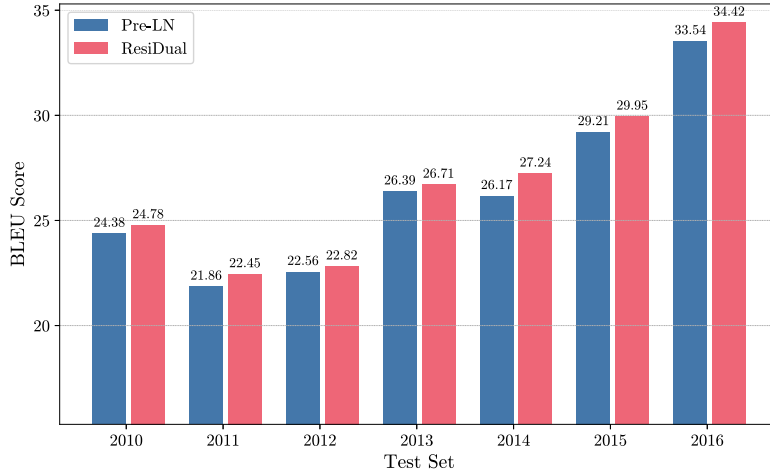


Fig. 4. Experimental results on WMT test sets of deep models with 50-layer encoder and 50-layer decoder.

different initialization schemes, amount to element-wise operations, which contribute negligibly to the overall cost compared with the dense matrix multiplications in the self-attention and feed-forward modules.

Within each depth setting, ResiDual remains in the same overall efficiency band as the other architectures, with only small differences relative to the standard Pre-LN and Post-LN baselines and essentially identical theoretical FLOPs. For instance, at E6D6 ResiDual attains 128.6k tokens/sec against 125.7k for Pre-LN, and at E18D18 it completes an epoch in 1324.6s against 1270.8s for Pre-LN. Peak memory consumption is likewise on par with the other variants. These results confirm that the dual residual connections in ResiDual resolve the theoretical optimization issues of Post-LN and Pre-LN without materially increasing training cost.

5.6. Empirical validation of the theoretical analysis

In this subsection, we empirically validate the theoretical claims developed in Section 4 through two complementary ablations. The first probes the robustness of each model to the learning-rate schedule, testing the practical implication of the gradient analysis that ResiDual should require less aggressive warm-up than Post-LN. The second tests whether the simplifications adopted in our proofs (zero-initialized W_Q , single-head attention, linearized MLP) materially alter the empirical conclusions.

Robustness to Learning-Rate Warm-Up. Learning-rate warm-up is a standard technique for stabilizing training, as discussed in Section 3.1. Because our approach aims to stabilize Transformer training, we evaluate several learning-rate schedules on IWSLT to study how the schedule affects optimization.

Specifically, denote the peak learning rate as η , the warm-up step as N_w , the total training step as N , and the learning rate at step t as $\alpha(t)$; we studied three types of learning rate schedulers as shown below:

- The traditional scheduler that initiates with a linear warm-up, succeeded by an inverse-square-root (ISR) decay:

$$\alpha(t) = \begin{cases} \frac{t}{N_w} \eta & t < N_w \\ \sqrt{\frac{N_w}{t}} \eta & t \geq N_w \end{cases}$$

- A scheduler that omits the learning rate warm-up stage and directly applies the square root (SR) decay from the peak learning rate:

$$\alpha(t) = \sqrt{1 - \frac{t}{N}} \eta$$

- A scheduler that omits the learning rate warm-up stage and immediately implements the linear decay from the peak learning rate:

$$\alpha(t) = \left(1 - \frac{t}{N}\right) \eta$$

Table 9 presents the results of different models and learning rate schedulers. The table includes results from both E6D6 and E12D12 models. Observations from the top three lines of the table indicate that the Post-LN model struggles with convergence. When learning rate warm-up is disabled or the model is scaled to deeper layers, convergence fails. The middle section of the table reveals that when learning rate warm-up is removed, the Pre-LN model, although capable of convergence, exhibits inferior results. The BLEU score decreases from about 35 points to 31 or 32 points. Lastly, our model not only converges, but also maintains competitive performance without learning-rate warm-up. This indicates that our model is easier to optimize in this setting.

Table 8

Efficiency analysis on the WMT dataset (means and standard deviations over 3 seeds; Fairseq AMP, no activation checkpointing). FLOPs denotes the theoretical forward-pass FLOPs per token (so the value is independent of batch size and sequence length), computed following standard counting conventions [1,44,45]; see Appendix A for the full derivation. **Within each depth block, ResiDual stays in the same overall efficiency band as the baselines, with only small differences relative to Pre-LN/Post-LN, essentially identical theoretical FLOPs, and peak memory comparable to the other variants.** Cross-block numbers use different training configurations and are not directly comparable.

Method	tokens/s ($\times 10^3$)		steps/s		wall-clock (s)		peak mem (GB)		FLOPs
<i>E6D6 Models</i>									
Pre-LN	125.7	0.9	0.270	0.002	909.6	6.3	73.8	0.5	2.13e8
Post-LN	132.9	1.2	0.272	0.002	898.4	7.9	77.2	0.5	2.13e8
DLCL	116.6	1.0	0.238	0.002	1023.6	8.2	78.0	0.5	2.14e8
T-Fixup	133.8	1.0	0.275	0.002	888.8	6.7	75.4	0.5	2.13e8
DeepNet	129.7	0.8	0.265	0.002	920.6	5.4	77.2	0.5	2.13e8
Admin	125.5	1.1	0.256	0.002	951.4	8.3	78.2	0.5	2.13e8
B2T	129.7	0.8	0.265	0.002	920.6	5.8	77.2	0.5	2.13e8
RealFormer	127.9	1.1	0.264	0.002	933.0	8.1	75.3	0.6	2.13e8
HybridNorm	136.9	1.7	0.280	0.003	872.0	10.5	76.2	0.5	2.14e8
Gemma 2	118.5	1.1	0.242	0.002	1007.8	9.2	78.4	0.5	2.14e8
ResiDual	128.6	1.6	0.265	0.003	928.4	11.7	75.8	0.5	2.13e8
<i>E18D18 Models</i>									
Pre-LN	93.9	0.7	0.102	0.001	1270.8	8.8	62.2	0.4	5.06e8
Post-LN	94.0	0.5	0.102	0.001	1269.4	6.9	63.6	0.4	5.06e8
DLCL	88.8	0.2	0.097	0.000	1334.0	5.2	64.4	0.5	5.07e8
T-Fixup	98.2	0.3	0.107	0.000	1215.2	3.9	60.1	0.4	5.06e8
DeepNet	93.7	0.1	0.102	0.000	1274.0	1.8	66.1	0.5	5.06e8
Admin	89.2	0.6	0.097	0.001	1337.8	9.2	68.1	0.5	5.06e8
B2T	94.1	0.6	0.102	0.001	1268.6	8.4	63.7	0.4	5.06e8
RealFormer	91.1	0.3	0.100	0.000	1310.4	4.9	67.6	0.4	5.06e8
HybridNorm	86.4	0.2	0.094	0.000	1316.0	7.8	69.0	0.5	5.07e8
Gemma 2	84.5	0.7	0.092	0.001	1412.8	11.0	68.2	0.5	5.07e8
ResiDual	90.1	0.5	0.098	0.001	1324.6	7.1	66.3	0.5	5.06e8

Table 9

Experimental results on IWSLT with different learning rate schedulers and model variants.

Method	Learning-Rate Scheduler		E6D6	E12D12
	Warm-up	Decay Formula		
Post-LN	Yes	ISR	35.37	Fail
Post-LN	No	SR	Fail	Fail
Post-LN	No	Linear	Fail	Fail
Pre-LN	Yes	ISR	35.12	35.18
Pre-LN	No	SR	32.28	31.82
Pre-LN	No	Linear	32.26	31.85
ResiDual	Yes	ISR	35.63	36.09
ResiDual	No	SR	35.76	35.57
ResiDual	No	Linear	35.96	35.72

Robustness to the Proof Assumptions. We next examine the three simplifications at initialization used in Sections 4.1 and 4.3: zero-initialized W_Q , single-head attention, and a linearized MLP. To verify that these assumptions do not limit the applicability of our method in practice, we evaluate all three configurations on the WMT dataset using E6D6 models.

We report the average BLEU, COMET (wmt22-comet-da, scaled by 100), and ChrF++ scores across the WMT test sets from 2010 to 2016. As shown in Table 10, ResiDual consistently outperforms the Pre-LN baseline in every configuration. This indicates that, although these simplifications make the derivations tractable, they do not compromise the central claims regarding the benefits of the dual-residual design: ResiDual retains stable gradients and preserves representation diversity, which translates into improved end-task performance. A per-year breakdown across all models and metrics is provided in Appendix G.

Table 10

Ablation study on theoretical assumptions. We report average BLEU, COMET (wmt22-comet-da, scaled by 100), and ChrF++ scores on the WMT test sets (2010–2016) using E6D6 models.

Setting	Model	BLEU	COMET	ChrF++
Zero-init W_O	Baseline	26.35	81.94	52.92
	ResiDual	26.73	82.88	53.10
Single-head attention	Baseline	24.44	78.89	51.13
	ResiDual	24.94	80.41	51.66
Linearized MLP	Baseline	26.44	82.01	52.94
	ResiDual	26.83	82.86	53.21

6. Conclusion

This work advances Transformer architecture design and offers an effective strategy for improving optimization and performance. We first examine the key limitations of two widely used variants: gradient vanishing in Post-LN and representation collapse in Pre-LN. We then introduce ResiDual, a novel architecture that mitigates both issues. ResiDual employs dual residual connections to address gradient vanishing and representation collapse simultaneously. Theoretical analysis and empirical results across multiple benchmarks and modalities validate that the proposed model overcomes both challenges while preserving the advantages of each variant. These results highlight the potential of ResiDual for future advances in Transformer architecture design and optimization.

CRedit authorship contribution statement

Shufang Xie: Writing – original draft, Visualization, Software, Data curation, Conceptualization; **Huishuai Zhang:** Formal analysis, Conceptualization; **Junliang Guo:** Software; **Xu Tan:** Writing – review & editing, Supervision, Conceptualization; **Jiang Bian:** Project administration; **Hany Hassan Awadalla:** Project administration; **Arul Menezes:** Project administration; **Tao Qin:** Supervision, Project administration; **Rui Yan:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All data are from public datasets. The code link is available in the article.

Appendix A. Details on FLOPs Computation

We estimate the theoretical floating-point operations (FLOPs) *per token* for the forward pass following standard analytical counting conventions [1,44,45], where one multiply-add operation counts as two FLOPs. Reporting per token makes the cost comparable across models trained with different batch sizes or sequence lengths. The vast majority of operations occur in the dense matrix multiplications of the multi-head attention and feed-forward networks (FFN).

Specifically, for sequence length S , model dimension d , and FFN inner dimension f , the per-token operations of a single encoder layer are as follows:

- **QKV Projections:** $3 \times (2d^2) = 6d^2$
- **Attention Scores & Application:** $2Sd + 2Sd = 4Sd$
- **Output Projection:** $2d^2$
- **FFN:** $2df + 2fd = 4df$

Minor element-wise operations such as Layer Normalization ($10d$) and residual additions ($2d$) are also incorporated but contribute negligibly to the overall count. Decoder layers follow a similar pattern, additionally accounting for cross-attention over the encoder outputs ($4Sd$ per decoder token for scores and application, plus projections). The total backward-pass FLOPs are approximated as twice the forward-pass FLOPs, leading to a per-token training cost of approximately $3\times$ the values reported in Table 8.

Appendix B. Proof of Theorem 1

Proof. For the Post-LN Transformer, the gradient of the parameters in the k -th layer (take W_k as an example) can be written as

$$\frac{\partial \mathcal{L}}{\partial w_k} = \frac{\partial \mathcal{L}}{\partial \bar{x}_{N+1}^{ln}} \frac{\partial \bar{x}_{N+1}^{ln}}{\partial \bar{x}_N^{ln}} \left(\prod_{l=k}^{N-1} \frac{\partial \bar{x}_{l+1}^a}{\partial \bar{x}_{l+1}^{ln}} \frac{\partial \bar{x}_{l+1}^{ln}}{\partial \bar{x}_l^a} \right) \frac{\partial \bar{x}_k^f}{\partial w_k}$$

$$= \frac{\partial \mathcal{L}}{\partial \bar{\mathbf{x}}_{N+1}^{ln}} \frac{\partial \bar{\mathbf{x}}_{N+1}^{ln}}{\partial \bar{\mathbf{x}}_N^a} \left(\prod_{l=k}^{N-1} \left(\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_{l+1}^f}{\partial \bar{\mathbf{x}}_{l+1}^{ln}} \right) \frac{\partial \bar{\mathbf{x}}_{l+1}^{ln}}{\partial \bar{\mathbf{x}}_l^a} \right) \frac{\partial \bar{\mathbf{x}}_k^f}{\partial \mathbf{w}_k}.$$

We care about the spectral norm of the term

$$\frac{\partial \bar{\mathbf{x}}_{N+1}^{ln}}{\partial \bar{\mathbf{x}}_N^a} \left(\prod_{l=k}^{N-1} \left(\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_{l+1}^f}{\partial \bar{\mathbf{x}}_{l+1}^{ln}} \right) \frac{\partial \bar{\mathbf{x}}_{l+1}^{ln}}{\partial \bar{\mathbf{x}}_l^a} \right),$$

which varies for different blocks.

For the feed-forward layer and attention layer, we respectively have $l \in [N]$,

$$\frac{\partial \bar{\mathbf{x}}_l^f}{\partial \bar{\mathbf{x}}_l^{ln}} = \begin{pmatrix} \mathbf{W}_l^T & & \\ & \ddots & \\ & & \mathbf{W}_l^T \end{pmatrix}$$

and

$$\frac{\partial \bar{\mathbf{x}}_l^f}{\partial \bar{\mathbf{x}}_l^{ln}} = \begin{pmatrix} \frac{1}{n} \mathbf{W}_{V,l}^T & \cdots & \frac{1}{n} \mathbf{W}_{V,l}^T \\ \vdots & \ddots & \vdots \\ \frac{1}{n} \mathbf{W}_{V,l}^T & \cdots & \frac{1}{n} \mathbf{W}_{V,l}^T \end{pmatrix},$$

based on the setup of the feed-forward layer and attention layer at the initialization. For the Layer Normalization layer, we have

$$\begin{aligned} \frac{\partial \bar{\mathbf{x}}_{l+1}^{ln}}{\partial \bar{\mathbf{x}}_l^a} &= \begin{pmatrix} \frac{\partial \text{LN}(\mathbf{x}_l^{a(1)})}{\partial \mathbf{x}_l^{a(1)}} & & \\ & \ddots & \\ & & \frac{\partial \text{LN}(\mathbf{x}_l^{a(n)})}{\partial \mathbf{x}_l^{a(n)}} \end{pmatrix} \\ &= \begin{pmatrix} \frac{\sqrt{d}}{\|\mathbf{x}_l^{a(1)}\|_2} \mathbf{I} & & \\ & \ddots & \\ & & \frac{\sqrt{d}}{\|\mathbf{x}_l^{a(n)}\|_2} \mathbf{I} \end{pmatrix}, \end{aligned}$$

as we assume on the Jacobian of Layer Normalization.

We note that $\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_l^f}{\partial \bar{\mathbf{x}}_l^{ln}}$ are block-circulant matrices for all l and the product of block-circulant matrices is also block-circulant. We know a block-circulant matrix has the following property

$$\left\| \begin{pmatrix} \mathbf{B} & \mathbf{A} & \cdots & \mathbf{A} \\ \mathbf{A} & \mathbf{B} & \cdots & \mathbf{A} \\ \vdots & & \ddots & \vdots \\ \mathbf{A} & \cdots & \mathbf{A} & \mathbf{B} \end{pmatrix} \right\|_2 = \|\mathbf{B} + (n-1)\mathbf{A}\|_2,$$

where $\mathbf{B}$ and $\mathbf{A}$ are square matrices and there are $n-1$ $\mathbf{A}$ s each row. Hence we have

$$\begin{aligned} &\left\| \frac{\partial \bar{\mathbf{x}}_{N+1}^{ln}}{\partial \bar{\mathbf{x}}_N^a} \left(\prod_{l=k}^{N-1} \left(\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_{l+1}^f}{\partial \bar{\mathbf{x}}_{l+1}^{ln}} \right) \frac{\partial \bar{\mathbf{x}}_{l+1}^{ln}}{\partial \bar{\mathbf{x}}_l^a} \right) \right\|_2 \\ &= \left(\prod_{l=k}^N \frac{\sqrt{d}}{\|\mathbf{x}_l^{a(i)}\|_2} \right) \left\| \left(\prod_{l=k+1}^N (\mathbf{I} + \mathbf{w}_l^T) \right) \right\|_2, \end{aligned}$$

where $\mathbf{w}_l$ represents either $\mathbf{W}_{V,l}$ or $\mathbf{W}_l$. We know that with high probability, $\|\mathbf{x}_l^{a(i)}\|_2 \in (1 \pm \epsilon)\sqrt{2d}$ where ϵ is a small positive constant, based on the assumption $\|\mathbf{x}_l^{ln(i)}\|_2 = \sqrt{d}$ and the random initialization of $\mathbf{w}_l$ for all $i \in [n]$. Thus we have a term $\left(\prod_{l=k}^N \frac{\sqrt{d}}{\|\mathbf{x}_l^{a(i)}\|_2} \right) \approx \mathcal{O}((1/2)^{(N-k)/2})$. Moreover, based on the random matrix argument [18], we have with high probability,

$$\left\| \prod_{l=k}^N (\mathbf{I} + \mathbf{w}_l^T) \right\|_2 \approx \mathcal{O}(e^{\sqrt{N-k}}).$$

Therefore, we have $\|\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k}\|_F \approx \mathcal{O}((1/2)^{(N-k)/2} \cdot e^{\sqrt{N-k}})$, which diminishes exponentially as $N-k$ is large.

On the other hand, we have the bound for Pre-LN transformer as follows.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \bar{\mathbf{x}}_{N+1}^a} \left(\prod_{l=k+1}^N \frac{\partial \bar{\mathbf{x}}_{l+1}^a}{\partial \bar{\mathbf{x}}_l^a} \right) \frac{\partial \bar{\mathbf{x}}_k^f}{\partial \mathbf{w}_k} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \bar{\mathbf{x}}_{N+1}^a} \left(\prod_{l=k+1}^N \left(\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_l^f}{\partial \bar{\mathbf{x}}_l^{ln}} \frac{\partial \bar{\mathbf{x}}_l^{ln}}{\partial \bar{\mathbf{x}}_l^a} \right) \right) \frac{\partial \bar{\mathbf{x}}_k^f}{\partial \mathbf{w}_k} \end{aligned}$$

We know that with high probability, $\|\mathbf{x}_l^{a(i)}\|_2 \in (1 \pm \epsilon)\sqrt{ld}$ based on the assumption $\|\mathbf{x}_l^{ln(i)}\|_2 = \sqrt{d}$ and the random initialization of $\mathbf{w}_l$ for all $i \in [n]$. Hence $\frac{\partial \mathbf{x}_l^{ln}}{\partial \mathbf{x}_l^a} \approx 1/\sqrt{lI}$. Therefore, with high probability, we have

$$\left\| \prod_{l=k+1}^N \left(\mathbf{I} + \frac{\partial \bar{\mathbf{x}}_l^f}{\partial \bar{\mathbf{x}}_l^{ln}} \frac{\partial \bar{\mathbf{x}}_l^{ln}}{\partial \bar{\mathbf{x}}_l^a} \right) \right\|_2 \approx \left\| \prod_{l=k+1}^N \left(\mathbf{I} + \frac{1}{\sqrt{l}} \mathbf{w}_l^T \right) \right\|_2 = \mathcal{O}(\log(N-k)),$$

where the last inequality is based on the argument for the product of random matrices [18]. Therefore, by further being aware of $\|\frac{\partial \mathbf{y}}{\partial \bar{\mathbf{x}}_{N+1}^a}\|_2 \approx 1/\sqrt{N}$, we have $\|\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k}\| \approx \mathcal{O}(\frac{\log(N-k)}{\sqrt{N}})$, which scales with $\sqrt{N}$ inverse proportionally. $\square$

Appendix C. Study of Adam

Let $\alpha, \tau, \epsilon, \beta_1, \beta_2$ denote the learning rate, step, smoothing factor, first decay rate, and second decay rate, respectively, and let $\mathbf{w}^{(t)}, \mathbf{g}, \hat{\mathbf{m}}^{(t)}, \hat{\mathbf{v}}^{(t)}$ denote the parameters, gradients, and bias-corrected first and second moment estimates at step t . Meanwhile, we use $\mathbf{u}(\mathbf{g}^{(t)}) = \alpha \cdot \hat{\mathbf{m}}^{(t)} / (\sqrt{\hat{\mathbf{v}}^{(t)}} + \epsilon)$ to denote the Adam update (i.e., $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \mathbf{u}(\mathbf{g}^{(t)})$), and the full formula is given below. Because the Adam [21] update is element-wise, we also use $u(g)$ to denote the scalar function of $\mathbf{u}(\mathbf{g})$, which means $\mathbf{u}(\mathbf{g}) = [u(g_1), u(g_2), \dots, u(g_d)]$. The Adam [21] optimization formula can be represented as

$$\begin{aligned} \mathbf{w}^{(t)} &\leftarrow \mathbf{w}^{(t-1)} - \alpha \cdot \hat{\mathbf{m}}^{(t)} / (\sqrt{\hat{\mathbf{v}}^{(t)}} + \epsilon) \\ \hat{\mathbf{m}}^{(t)} &\leftarrow \mathbf{m}^{(t)} / (1 - \beta_1^t), \quad \hat{\mathbf{v}}^{(t)} \leftarrow \mathbf{v}^{(t)} / (1 - \beta_2^t) \\ \mathbf{m}^{(t)} &\leftarrow \beta_1 \cdot \mathbf{m}^{(t-1)} + (1 - \beta_1) \cdot \mathbf{g} \\ \mathbf{v}^{(t)} &\leftarrow \beta_2 \cdot \mathbf{v}^{(t-1)} + (1 - \beta_2) \cdot \mathbf{g}^2. \end{aligned}$$

The absolute condition number $\hat{\kappa}$ for $\mathbf{u}(\mathbf{g}_t)$ is

$$\hat{\kappa} = \lim_{\delta \rightarrow 0} \sup_{\|\delta \mathbf{g}\| \leq \delta} \frac{\|\mathbf{u}(\mathbf{g} + \delta \mathbf{g}) - \mathbf{u}(\mathbf{g})\|}{\|\delta \mathbf{g}\|} = \sqrt{\sum_{i=1}^d \left(\frac{\partial u}{\partial g_i} \right)^2}.$$

The full expression of $u'(g_{t,i})$ is

$$\begin{aligned} \frac{\partial u}{\partial g} &= \frac{\alpha g \sqrt{\frac{\beta_2 v^{(t-1)} + g^2 \cdot (1 - \beta_2)}{1 - \beta_2^t}} (1 - \beta_2) (\beta_1 m^{(t-1)} + g(1 - \beta_1))}{(1 - \beta_1^t) \left(\epsilon + \sqrt{\frac{\beta_2 v^{(t-1)} + g^2 \cdot (1 - \beta_2)}{1 - \beta_2^t}} \right)^2 (\beta_2 v^{(t-1)} + g^2 \cdot (1 - \beta_2))} \\ &\quad + \frac{\alpha(1 - \beta_1)}{(1 - \beta_1^t) \left(\epsilon + \sqrt{\frac{\beta_2 v^{(t-1)} + g^2 \cdot (1 - \beta_2)}{1 - \beta_2^t}} \right)}. \end{aligned}$$

Therefore, when the gradient $g = 0$, we have

$$\frac{\partial u}{\partial g} = \frac{\alpha(1 - \beta_1)}{(1 - \beta_1^t) \left(\epsilon + \sqrt{\frac{\beta_2 v_{l-1,i}^{(t-1)}}{1 - \beta_2^t}} \right)}.$$

In the early stage (i.e., t is small), for the vanished gradient ($g_i = 0$), the absolute condition number $\hat{\kappa}$ is

$$\hat{\kappa} = \alpha \frac{1 - \beta_1}{1 - \beta_1^t} \sqrt{\sum_{i=1}^d \frac{1}{\left(\epsilon + \sqrt{\frac{\beta_2 v_i^{(t-1)}}{1 - \beta_2^t}} \right)^2}} \approx \frac{\alpha \sqrt{d}}{\epsilon}. \quad (\text{C.1})$$

Intuitively, when there is a small noise $\|\delta \mathbf{g}\| \leq \delta$ added to the gradient $\mathbf{g}$, the change of the update $\|\mathbf{u}(\mathbf{g} + \delta \mathbf{g}) - \mathbf{u}(\mathbf{g})\|$ could be much larger than $\|\delta \mathbf{g}\|$.

To verify if the approximation is accurate, we simulate the Adam update and compute the $\hat{\kappa}$, we use the parameter as $d = 1024, \epsilon = 10^{-6}, \beta_1 = 0.9, \beta_2 = 0.98, \alpha = 10^{-4}$. Then for each step, we first random sample $\mathbf{g} \sim \mathcal{N}(0, \sigma_g^2 \mathbf{I})$, then compute the $\hat{\kappa}$ and momentums based on Adam update formulas. In Fig. C.5, we show the simulated results by sampling $\mathbf{g} \sim \mathcal{N}(0, \sigma_g^2 \mathbf{I})$ where σ_g ranges from 0 to 10^{-7} . In the left plot, we show how $\hat{\kappa}$ changes w.r.t. t for different σ_g . We can find that our estimation of $\hat{\kappa}$ is accurate, as most of the lines are overlapped. Besides, it also shows that even after 20 update steps, the $\hat{\kappa}$ is still greater than 300. As many lines overlap in the left plot, on the right side we show a zoomed-in view by selecting five timestamps and plotting $\hat{\kappa}$ w.r.t. σ_g . It is clear that $\hat{\kappa}$ takes large values when σ_g is small.

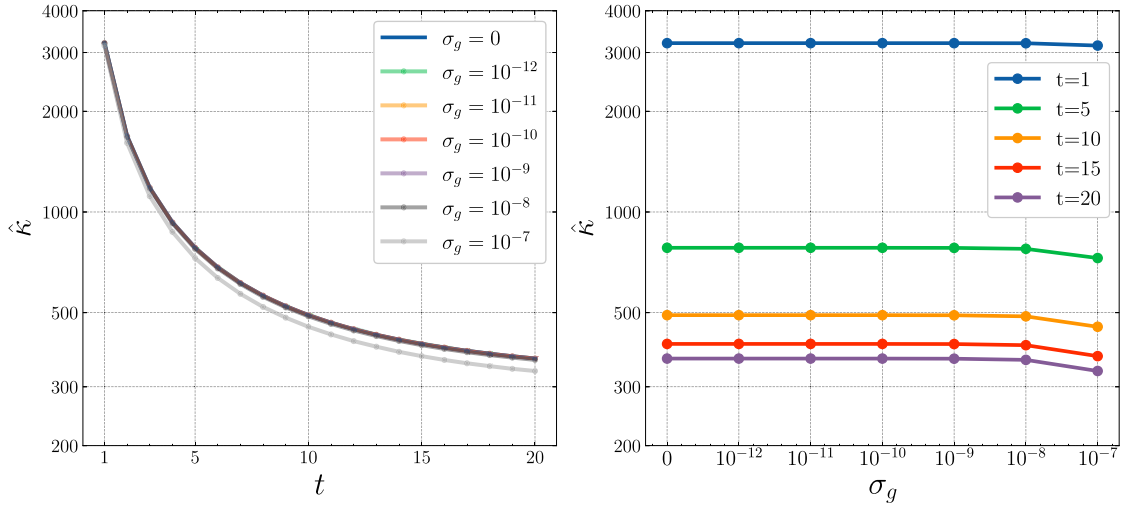


Fig. C.5. The absolute condition number $\hat{\kappa}$ of Adam in the early stage of model training, w.r.t. t (left figure) and σ_g (right figure).

Appendix D. Implementation Details

D.1. Data processing

The IWSLT dataset contains 140k sentence pairs. We followed the scripts in Fairseq [24] to preprocess the data Table F.16. The WMT dataset is larger, with 4M sentence pairs. We followed the preprocessing steps outlined in [28] by tokenizing the data with Moses tokenizer and then processing it with BPE [29]. The model was trained on the WMT-14 training set and evaluated on the test set from years 2010 to 2016, following [28]. The OPUS-100 dataset is a large-scale multilingual dataset containing 100 languages and approximately 55M sentence pairs. We used the script from [27] to tokenize the data and used SentencePiece [30] to segment the tokens. Because we train our model in both directions (to and from English), the total training data is about 110M.

The data processing scripts are listed below .

- **IWSLT:** <https://github.com/facebookresearch/fairseq/blob/main/examples/translation/prepare-iwslt14.sh>.
- **WMT:** <https://github.com/facebookresearch/fairseq/blob/main/examples/translation/prepare-wmt14en2de.sh>.
- **OPUS-100:** <https://github.com/bzhangGo/zero>.

D.2. Hyper-parameters

Table D.11
Hyper-parameters of different datasets training.

Parameter	IWSLT	WMT	OPUS-100
Dropout	0.3	0.3	0.1
Embedding dim	256	512	512
FFN dim	1024	2048	2048
Attention heads	4	8	8
Encoder layers	6/12	6/18	18
Decoder layers	6/12	6/18	18
Learning rate	$5 * 10^{-4}$	$1 * 10^{-3}$	$1 * 10^{-3}$
Learning rate scheduler	inverse sqrt	inverse sqrt	inverse sqrt
Warm-up steps	4000	4000	4000
Label smoothing	0.1	0.1	0.1
Weight decay	0.0001	0.0001	0.0001
Adam β	0.9, 0.98	0.9, 0.98	0.9, 0.98
Max update steps	300k	500k	100k

Table D.12
Hyper-parameters of ViT training (Table F.17).

Parameter	Value
Image size	384
Learning rate	0.01
Total steps	200k
Train batch size	64
Warm-up steps	500
LR scheduler	cosine
patch size	16/32
Depth	12
Hidden size	768
MLP size	3072
Heads	12

Appendix E. Details of Language Modeling Experiments

All language-modeling experiments use decoder-only Transformers with causal self-attention, trained under the standard autoregressive next-token prediction objective. For Gigaword, we fine-tune the pretrained language-model checkpoints and report ROUGE-1, ROUGE-2, and ROUGE-L on the summarization task (Table E.13).

Table E.13
Model architecture and training details for Language Modeling experiments.

Parameter	L12	L24
Depth (n_layer)	12	24
Model Dimension (n_embd)	768	768
Number of Heads (n_head)	6	6
Head Dimension	128	128
Number of KV Heads (n_kv_head)	6	6
Max Sequence Length	2048	2048
Total Training Tokens	$\approx 10\text{B}$	$\approx 10\text{B}$
Device Batch Size	32	32
Total Batch Size	131,072	524,288

For Gigaword summarization fine-tuning, we adopt the same recipe for the L12 and L24 models. We use the standard splits, with 3,803,957, 189,651, and 1,951 examples for training, validation, and test, respectively. Each article is wrapped in the prompt template

```
Write a short news headline.
Article: {article}
Return only the headline.
```

and the cross-entropy loss is computed on the headline tokens only (prompt tokens are masked out). Key hyper-parameters are listed in Table E.14; we evaluate with greedy decoding (temperature 0) Table F.18.

Table E.14
Gigaword summarization fine-tuning recipe (shared by L12 and L24).

Parameter	Value
Optimizer	AdamW
Peak learning rate	2×10^{-4}
Initial learning rate	2×10^{-5}
LR schedule	linear decay to 0
Weight decay	0
Precision	bfloat16
Epochs	1
Effective batch size	256 examples / update
Total updates	14,859 ($\lceil \text{train_rows} / 256 \rceil$)
Max sequence length	256
Decoding (test)	greedy, temperature 0

Appendix F. Detailed Machine Translation Metrics

In this section, we provide the per-year ChrF++ [31] and COMET [32] (wmt22-comet-da, scaled by 100) scores for all methods on the WMT dataset, complementing the BLEU results reported in Tables 5 and 6. All results are averaged over 3 random seeds; we report mean and standard deviation.

Table F.15

ChrF++ scores on WMT dataset (year 2010 to 2016) with E6D6 models. Mean $\pm$ standard deviation over 3 seeds.

Method	2010	2011	2012	2013	2014	2015	2016	Avg.
Pre-LN	51.03 $\pm$ 0.23	49.03 $\pm$ 0.08	48.61 $\pm$ 0.11	52.08 $\pm$ 0.05	54.73 $\pm$ 0.16	55.73 $\pm$ 0.23	59.09 $\pm$ 0.08	52.90 $\pm$ 0.13
Post-LN	51.23 $\pm$ 0.10	49.08 $\pm$ 0.28	49.02 $\pm$ 0.09	52.15 $\pm$ 0.27	55.02 $\pm$ 0.11	55.68 $\pm$ 0.06	58.91 $\pm$ 0.25	53.01 $\pm$ 0.16
DLCL	51.04 $\pm$ 0.13	49.18 $\pm$ 0.04	49.04 $\pm$ 0.19	52.20 $\pm$ 0.08	55.07 $\pm$ 0.09	56.01 $\pm$ 0.12	59.16 $\pm$ 0.03	53.10 $\pm$ 0.10
T-Fixup	51.22 $\pm$ 0.16	49.03 $\pm$ 0.13	48.81 $\pm$ 0.10	52.12 $\pm$ 0.16	54.42 $\pm$ 0.02	55.63 $\pm$ 0.16	59.35 $\pm$ 0.07	52.94 $\pm$ 0.12
DeepNet	51.20 $\pm$ 0.01	49.20 $\pm$ 0.07	49.09 $\pm$ 0.23	52.19 $\pm$ 0.15	54.95 $\pm$ 0.11	55.85 $\pm$ 0.01	59.01 $\pm$ 0.07	53.07 $\pm$ 0.09
Admin	51.15 $\pm$ 0.03	49.05 $\pm$ 0.02	48.99 $\pm$ 0.08	52.14 $\pm$ 0.19	55.04 $\pm$ 0.13	55.87 $\pm$ 0.17	58.81 $\pm$ 0.04	53.01 $\pm$ 0.09
B2T	51.10 $\pm$ 0.10	49.14 $\pm$ 0.17	49.07 $\pm$ 0.01	52.50$\pm$0.06	55.04 $\pm$ 0.18	55.97 $\pm$ 0.02	59.42$\pm$0.23	53.17 $\pm$ 0.11
RealFormer	51.04 $\pm$ 0.08	49.20 $\pm$ 0.14	48.93 $\pm$ 0.12	52.16 $\pm$ 0.04	55.00 $\pm$ 0.21	55.83 $\pm$ 0.14	58.87 $\pm$ 0.18	53.00 $\pm$ 0.13
HybridNorm	51.19 $\pm$ 0.15	49.15 $\pm$ 0.06	49.09 $\pm$ 0.05	52.17 $\pm$ 0.01	54.88 $\pm$ 0.04	55.77 $\pm$ 0.07	58.75 $\pm$ 0.24	53.00 $\pm$ 0.09
Gemma 2	51.11 $\pm$ 0.11	48.97 $\pm$ 0.15	48.77 $\pm$ 0.09	52.22 $\pm$ 0.10	54.76 $\pm$ 0.03	55.60 $\pm$ 0.15	59.00 $\pm$ 0.02	52.92 $\pm$ 0.09
ResiDual	51.22$\pm$0.05	49.22$\pm$0.04	49.10$\pm$0.02	52.23 $\pm$ 0.11	55.11$\pm$0.05	55.96$\pm$0.04	59.02 $\pm$ 0.35	53.12$\pm$0.09

Table F.16

ChrF++ scores on WMT dataset (year 2010 to 2016) with E18D18 models. Mean $\pm$ standard deviation over 3 seeds. Post-LN failed to converge.

Method	2010	2011	2012	2013	2014	2015	2016	Avg.
Pre-LN	51.42 $\pm$ 0.02	49.30 $\pm$ 0.17	49.28 $\pm$ 0.01	52.71 $\pm$ 0.07	55.75 $\pm$ 0.08	56.53 $\pm$ 0.05	59.10 $\pm$ 0.07	53.44 $\pm$ 0.07
DLCL	51.06 $\pm$ 0.00	49.24 $\pm$ 0.00	48.78 $\pm$ 0.00	52.23 $\pm$ 0.00	54.94 $\pm$ 0.00	55.86 $\pm$ 0.00	58.83 $\pm$ 0.00	52.99 $\pm$ 0.00
T-Fixup	51.08 $\pm$ 0.00	48.83 $\pm$ 0.08	48.72 $\pm$ 0.03	52.06 $\pm$ 0.02	54.65 $\pm$ 0.04	55.69 $\pm$ 0.06	59.21 $\pm$ 0.18	52.89 $\pm$ 0.06
DeepNet	51.14 $\pm$ 1.15	48.90 $\pm$ 1.18	48.85 $\pm$ 1.17	52.11 $\pm$ 1.46	54.81 $\pm$ 2.48	55.70 $\pm$ 2.07	58.69 $\pm$ 1.64	52.89 $\pm$ 1.60
Admin	50.76 $\pm$ 1.52	48.74 $\pm$ 1.57	48.59 $\pm$ 1.51	51.83 $\pm$ 1.87	54.37 $\pm$ 2.86	55.41 $\pm$ 2.54	58.42 $\pm$ 2.32	52.59 $\pm$ 2.03
B2T	50.39 $\pm$ 2.11	48.38 $\pm$ 1.93	48.31 $\pm$ 1.96	51.50 $\pm$ 2.26	54.05 $\pm$ 3.43	54.99 $\pm$ 3.12	57.94 $\pm$ 3.00	52.22 $\pm$ 2.55
RealFormer	51.32 $\pm$ 0.10	49.34 $\pm$ 0.04	49.58 $\pm$ 0.04	52.02 $\pm$ 0.21	55.24 $\pm$ 0.04	56.49 $\pm$ 0.19	59.14 $\pm$ 0.05	53.30 $\pm$ 0.10
HybridNorm	50.06 $\pm$ 0.89	47.97 $\pm$ 0.69	47.78 $\pm$ 0.83	50.88 $\pm$ 0.76	52.93 $\pm$ 1.31	54.10 $\pm$ 1.10	57.63 $\pm$ 0.98	51.62 $\pm$ 0.94
Gemma 2	51.15 $\pm$ 0.10	49.09 $\pm$ 0.05	49.01 $\pm$ 0.11	52.58 $\pm$ 0.16	55.72 $\pm$ 0.13	56.01 $\pm$ 0.05	58.89 $\pm$ 0.40	53.21 $\pm$ 0.14
ResiDual	51.58$\pm$0.12	49.70$\pm$0.10	49.67$\pm$0.09	52.94$\pm$0.12	55.85$\pm$0.21	56.66$\pm$0.24	59.25$\pm$0.12	53.66$\pm$0.14

Table F.17

COMET (wmt22-comet-da, scaled by 100) on WMT dataset (year 2010 to 2016) with E6D6 models. Mean $\pm$ standard deviation over 3 seeds. Apparent zero standard deviations arise when all seeds produce the same value after rounding to two decimal places.

Method	2010	2011	2012	2013	2014	2015	2016	Avg.
Pre-LN	81.81 $\pm$ 0.10	79.90 $\pm$ 0.14	80.05 $\pm$ 0.05	82.84 $\pm$ 0.01	83.18 $\pm$ 0.25	82.16 $\pm$ 0.00	83.35 $\pm$ 0.14	81.90 $\pm$ 0.10
Post-LN	82.71$\pm$0.10	80.94 $\pm$ 0.04	81.12$\pm$0.20	83.73$\pm$0.19	83.97 $\pm$ 0.06	83.10$\pm$0.01	84.04$\pm$0.04	82.80$\pm$0.09
DLCL	82.43 $\pm$ 0.08	80.69 $\pm$ 0.00	80.86 $\pm$ 0.08	83.55 $\pm$ 0.14	83.94 $\pm$ 0.04	82.95 $\pm$ 0.17	83.79 $\pm$ 0.07	82.60 $\pm$ 0.08
T-Fixup	82.13 $\pm$ 0.16	80.30 $\pm$ 0.24	80.41 $\pm$ 0.26	83.24 $\pm$ 0.22	83.10 $\pm$ 0.17	82.16 $\pm$ 0.13	83.37 $\pm$ 0.09	82.10 $\pm$ 0.18
DeepNet	82.56 $\pm$ 0.09	80.74 $\pm$ 0.17	80.87 $\pm$ 0.27	83.53 $\pm$ 0.23	83.75 $\pm$ 0.13	82.86 $\pm$ 0.04	83.82 $\pm$ 0.02	82.59 $\pm$ 0.14
Admin	82.36 $\pm$ 0.07	80.51 $\pm$ 0.01	80.73 $\pm$ 0.03	83.44 $\pm$ 0.07	83.65 $\pm$ 0.01	82.69 $\pm$ 0.08	83.72 $\pm$ 0.01	82.44 $\pm$ 0.04
B2T	82.46 $\pm$ 0.22	80.77 $\pm$ 0.18	80.92 $\pm$ 0.08	83.58 $\pm$ 0.03	83.90 $\pm$ 0.03	82.95 $\pm$ 0.06	83.91 $\pm$ 0.11	82.64 $\pm$ 0.10
RealFormer	82.37 $\pm$ 0.01	80.63 $\pm$ 0.14	80.70 $\pm$ 0.10	83.39 $\pm$ 0.05	83.75 $\pm$ 0.12	82.69 $\pm$ 0.08	83.74 $\pm$ 0.08	82.47 $\pm$ 0.08
HybridNorm	82.46 $\pm$ 0.17	80.58 $\pm$ 0.17	80.84 $\pm$ 0.07	83.55 $\pm$ 0.03	83.80 $\pm$ 0.02	82.88 $\pm$ 0.11	83.86 $\pm$ 0.10	82.57 $\pm$ 0.09
Gemma 2	81.91 $\pm$ 0.23	79.95 $\pm$ 0.22	80.25 $\pm$ 0.16	82.98 $\pm$ 0.25	83.27 $\pm$ 0.07	82.24 $\pm$ 0.11	83.31 $\pm$ 0.09	81.99 $\pm$ 0.16
ResiDual	82.71$\pm$0.02	81.08$\pm$0.13	81.12$\pm$0.06	83.73$\pm$0.01	84.02$\pm$0.09	83.01 $\pm$ 0.10	83.94 $\pm$ 0.13	82.80$\pm$0.08

Table F.18

COMET (wmt22-comet-da, scaled by 100) on WMT dataset (year 2010 to 2016) with E18D18 models. Mean $\pm$ standard deviation over 3 seeds. Post-LN failed to converge.

Method	2010	2011	2012	2013	2014	2015	2016	Avg.
Pre-LN	82.73 $\pm$ 0.29	80.62 $\pm$ 0.32	80.74 $\pm$ 0.21	83.71 $\pm$ 0.23	84.31 $\pm$ 0.22	83.47 $\pm$ 0.23	84.19 $\pm$ 0.07	82.82 $\pm$ 0.23
DLCL	82.52 $\pm$ 0.13	80.58 $\pm$ 2.03	80.87 $\pm$ 2.12	83.54 $\pm$ 0.40	84.05 $\pm$ 0.38	82.90 $\pm$ 1.67	84.15 $\pm$ 0.23	82.66 $\pm$ 0.99
T-Fixup	81.61 $\pm$ 0.12	79.46 $\pm$ 0.28	79.68 $\pm$ 0.26	82.66 $\pm$ 0.14	82.93 $\pm$ 0.19	81.81 $\pm$ 0.22	83.02 $\pm$ 0.08	81.60 $\pm$ 0.18
DeepNet	82.07 $\pm$ 2.12	80.05 $\pm$ 2.13	80.44 $\pm$ 2.12	82.98 $\pm$ 2.21	83.38 $\pm$ 2.77	82.46 $\pm$ 3.01	83.39 $\pm$ 2.52	82.11 $\pm$ 2.41
Admin	81.43 $\pm$ 2.67	79.62 $\pm$ 2.46	79.89 $\pm$ 2.42	82.51 $\pm$ 2.72	82.80 $\pm$ 3.42	82.08 $\pm$ 3.51	82.81 $\pm$ 2.97	81.59 $\pm$ 2.88
B2T	80.96 $\pm$ 3.22	79.39 $\pm$ 2.87	79.44 $\pm$ 2.99	82.15 $\pm$ 2.98	82.48 $\pm$ 3.70	81.63 $\pm$ 3.80	82.40 $\pm$ 3.43	81.21 $\pm$ 3.28
RealFormer	83.22 $\pm$ 0.07	81.35 $\pm$ 0.18	81.61 $\pm$ 0.23	84.25 $\pm$ 0.06	84.90 $\pm$ 0.18	83.98 $\pm$ 0.21	84.75 $\pm$ 0.11	83.43 $\pm$ 0.15
HybridNorm	79.76 $\pm$ 1.31	78.02 $\pm$ 1.20	78.02 $\pm$ 1.31	80.93 $\pm$ 2.42	80.82 $\pm$ 0.35	79.69 $\pm$ 1.47	81.05 $\pm$ 1.23	79.76 $\pm$ 1.33
Gemma 2	82.54 $\pm$ 0.46	80.58 $\pm$ 0.53	80.81 $\pm$ 0.39	83.82 $\pm$ 0.35	84.52 $\pm$ 0.32	83.61 $\pm$ 0.24	84.32 $\pm$ 0.28	82.89 $\pm$ 0.37
ResiDual	83.43$\pm$0.04	81.54$\pm$0.22	81.84$\pm$0.16	84.37$\pm$0.01	84.98$\pm$0.03	84.23$\pm$0.14	84.77$\pm$0.12	83.59$\pm$0.10

Appendix G. Detailed Results for Ablation Studies

In this section, we present the comprehensive per-year evaluation results for our ablation studies on the theoretical assumptions (zero-initialized W_O , single-head attention, and linearized MLP). We report BLEU, COMET (wmt22-comet-da, scaled by 100), and ChrF++ scores for the individual WMT translation tasks from 2010 to 2016. As shown in Table G.19, our proposed ResiDual architecture exhibits robust performance improvements over the Baseline (Pre-LN) across almost all individual test sets and evaluation metrics.

Table G.19

Detailed ablation study results on theoretical assumptions using WMT test sets (2010–2016) with E6D6 models.

Setting	Model	2010	2011	2012	2013	2014	2015	2016	Average
<i>BLEU</i>									
Zero-init W_O	Baseline	24.18	22.09	22.22	25.88	26.30	29.38	34.39	26.35
	ResiDual	24.36	22.28	22.59	26.33	27.25	30.08	34.25	26.73
Single-head attention	Baseline	22.63	20.41	20.74	24.49	23.73	27.13	31.93	24.44
	ResiDual	23.01	20.68	20.89	24.69	24.80	27.80	32.73	24.94
Linearized MLP	Baseline	24.25	21.78	22.19	26.19	26.55	29.87	34.23	26.44
	ResiDual	24.45	22.25	22.57	26.66	27.27	30.07	34.55	26.83
<i>COMET (wmt22-comet-da, scaled by 100)</i>									
Zero-init W_O	Baseline	81.80	80.17	80.15	82.88	83.12	82.26	83.21	81.94
	ResiDual	82.81	81.07	81.19	83.81	84.10	83.29	83.91	82.88
Single-head attention	Baseline	78.97	77.40	77.29	80.51	79.71	78.52	79.81	78.89
	ResiDual	80.53	78.74	78.82	81.70	81.43	80.21	81.47	80.41
Linearized MLP	Baseline	81.92	80.13	80.17	83.10	83.23	82.11	83.39	82.01
	ResiDual	82.71	81.07	81.23	83.84	83.98	83.09	84.06	82.86
<i>ChrF++</i>									
Zero-init W_O	Baseline	51.08	49.09	48.78	51.97	54.66	55.59	59.25	52.92
	ResiDual	51.18	49.15	49.03	52.32	55.01	55.98	59.01	53.10
Single-head attention	Baseline	49.73	47.63	47.15	50.68	52.21	53.51	57.01	51.13
	ResiDual	50.15	47.94	47.67	51.01	53.10	54.11	57.62	51.66
Linearized MLP	Baseline	51.07	48.97	48.63	52.18	54.76	55.81	59.18	52.94
	ResiDual	51.29	49.22	49.07	52.44	55.16	55.99	59.30	53.21

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017) 5998–6008.
- [2] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, Improving language understanding by generative pre-training, (2018). Technical report, OpenAI. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877–1901.
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: transformers for image recognition at scale, in: ICLR, 2021, pp. 611–631.
- [5] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, T.-Y. Liu, FastSpeech: fast, robust and controllable text to speech, Adv. Neural Inf. Process. Syst. 32 (2019) 3165–3174.

- [6] L. Liu, X. Liu, J. Gao, W. Chen, J. Han, Understanding the difficulty of training transformers, in: 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Association for Computational Linguistics (ACL), 2020, pp. 5747–5763.
- [7] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, T.-Y. Liu, On layer normalization in the transformer architecture, in: International Conference on Machine Learning, PMLR, 2020, pp. 10524–10533.
- [8] T. Bachlechner, B.P. Majumder, H. Mao, G. Cottrell, J. McAuley, Zero is all you need: fast convergence at large depth, in: Uncertainty in Artificial Intelligence, PMLR, 2021, pp. 1352–1361.
- [9] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Minneapolis, Minnesota, 2019, pp. 4171–4186.
- [10] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: a robustly optimized BERT pretraining approach, (2019). [arXiv:1907.11692](https://arxiv.org/abs/1907.11692)
- [11] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al., Language models are unsupervised multitask learners, OpenAI Blog 1 (8) (2019) 9.
- [12] Q. Wang, B. Li, T. Xiao, J. Zhu, C. Li, D.F. Wong, L.S. Chao, Learning deep transformer models for machine translation, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 1810–1822.
- [13] S. Shleifer, J. Weston, M. Ott, NormFormer: improved transformer pretraining with extra normalization, (2021). [arXiv:2110.09456](https://arxiv.org/abs/2110.09456) [cs.CL]
- [14] R. He, A. Ravula, B. Kanagal, J. Ainslie, RealFormer: transformer likes residual attention, in: Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, Association for Computational Linguistics, 2021, pp. 929–943. <https://doi.org/10.18653/v1/2021.findings-acl.81>
- [15] S. Takase, S. Kiyono, S. Kobayashi, J. Suzuki, B2T connection: serving stability and performance in deep transformers, in: Findings of the Association for Computational Linguistics: ACL 2023, Association for Computational Linguistics, 2023, pp. 3078–3095. <https://doi.org/10.18653/v1/2023.findings-acl.192>
- [16] P. Li, L. Yin, S. Liu, Mix-LN: unleashing the power of deeper layers by combining Pre-LN and Post-LN, in: International Conference on Learning Representations, 2025, pp. 98244–98258.
- [17] H. Wang, S. Ma, L. Dong, S. Huang, D. Zhang, F. Wei, DeepNet: scaling transformers to 1,000 layers, IEEE Trans. Pattern Anal. Mach. Intell. 46 (10) (2024) 6761–6774. <https://doi.org/10.1109/TPAMI.2024.3386927>
- [18] H. Zhang, D. Yu, M. Yi, W. Chen, T.-Y. Liu, Stabilize deep ResNet with a sharp scaling factor τ , Mach. Learn. 111 (9) (2022) 3359–3392.
- [19] X.S. Huang, F. Perez, J. Ba, M. Volkovs, Improving transformer optimization through better initialization, in: International Conference on Machine Learning, PMLR, 2020, pp. 4475–4483.
- [20] H. Wang, S. Ma, S. Huang, L. Dong, W. Wang, Z. Peng, Y. Wu, P. Bajaj, S. Singhal, A. Benhaim, B. Patra, Z. Liu, V. Chaudhary, X. Song, F. Wei, Magneto: a foundation transformer, in: International Conference on Machine Learning, PMLR, 2023, pp. 36077–36092.
- [21] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, in: International Conference on Learning Representations, 2015.
- [22] Z. Zhuo, Y. Zeng, Y. Wang, S. Zhang, X. Li, J. Yang, X. Zhou, J. Ma, HybridNorm: towards stable and efficient transformer training via hybrid normalization, in: Advances in Neural Information Processing Systems, 38, 2025, pp. 25106–25147.
- [23] G. Team, et al., Gemma 2: improving open language models at a practical size, (2024). [arXiv:2408.00118](https://arxiv.org/abs/2408.00118)
- [24] M. Ott, S. Edunov, A. Baevski, A. Fan, S. Gross, N. Ng, D. Grangier, M. Auli, fairseq: a fast, extensible toolkit for sequence modeling, in: Proceedings of NAACL-HLT 2019: Demonstrations, 2019.
- [25] M. Cettolo, J. Niehues, S. Stüker, L. Bentivogli, M. Federico, Report on the 11th IWSLT evaluation campaign, IWSLT 2014, in: Proceedings of the 11th International Workshop on Spoken Language Translation, Lake Tahoe, California, USA, 2014, pp. 2–17.
- [26] O. Bojar, C. Buck, C. Federmann, B. Haddow, P. Koehn, J. Leveling, C. Monz, P. Pecina, M. Post, H. Saint-Amand, et al., Findings of the 2014 workshop on statistical machine translation, in: Proceedings of the Ninth Workshop on Statistical Machine Translation, 2014, pp. 12–58.
- [27] B. Zhang, P. Williams, I. Titov, R. Sennrich, Improving massively multilingual neural machine translation and zero-shot translation, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Online, 2020, pp. 1628–1639. <https://doi.org/10.18653/v1/2020.acl-main.148>
- [28] S. Takase, S. Kiyono, Rethinking perturbations in encoder-decoders for fast training, Proc. 2021 Conf. N. Am. Chapter Assoc. Comput. Linguist.: Hum. Lang. Technol. (2021). <https://doi.org/10.18653/v1/2021.naacl-main.460>
- [29] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Berlin, Germany, 2016, pp. 1715–1725.
- [30] T. Kudo, J. Richardson, SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2018, pp. 66–71.
- [31] M. Popović, chrF++: words helping character n-grams, in: Proceedings of the Second Conference on Machine Translation, 2017, pp. 612–618.
- [32] R. Rei, C. Stewart, A.C. Farinha, A. Lavie, COMET: a neural framework for MT evaluation, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 2685–2702.
- [33] P. Koehn, Statistical significance tests for machine translation evaluation, in: Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2004, pp. 388–395.
- [34] S. Riezler, J.T. Maxwell, III, On some pitfalls in automatic evaluation and significance testing for MT, in: Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization, 2005, pp. 57–64.
- [35] A. Gokaslan, V. Cohen, OpenWebText corpus, (2019). <http://Skylion007.github.io/OpenWebTextCorpus>
- [36] T. Mihaylov, P. Clark, T. Khot, A. Sabharwal, Can a suit of armor conduct electricity? a new dataset for open book question answering, in: EMNLP, 2018.
- [37] A. Talmor, J. Herzig, N. Lourie, J. Berant, CommonsenseQA: a question answering challenge targeting commonsense knowledge, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019.
- [38] Y. Bisk, R. Zellers, R. Le Bras, J. Gao, Y. Choi, PIQA: reasoning about physical commonsense in natural language, in: AAAI, 2020.
- [39] K. Sakaguchi, R. Le Bras, C. Bhagavatula, Y. Choi, WinoGrande: an adversarial winograd schema challenge at scale, Commun. ACM 64 (9) (2021) 99–106.
- [40] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, O. Tafjord, Think you have solved question answering? try arc, the ai2 reasoning challenge, (2018). [arXiv:1803.05457](https://arxiv.org/abs/1803.05457)
- [41] A.M. Rush, S. Chopra, J. Weston, A neural attention model for abstractive sentence summarization, in: Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2015, pp. 379–389. <https://doi.org/10.18653/v1/D15-1044>
- [42] D. Graff, J. Kong, K. Chen, K. Maeda, English gigaword, Linguist. Data Consort. Phila. 4 (1) (2003) 34. <https://www.tensorflow.org/datasets/catalog/gigaword>
- [43] E.D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, Q.V. Le, AutoAugment: learning augmentation strategies from data, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019.
- [44] J. Kaplan, S. McCandlish, T. Henighan, T.B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling laws for neural language models, (2020). [arXiv:2001.08361](https://arxiv.org/abs/2001.08361)
- [45] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L.A. Hendricks, J. Welbl, A. Clark, et al., Training compute-optimal large language models, in: Advances in Neural Information Processing Systems, 35, 2022, pp. 30016–30030.